

Angular Learning Series

Topics 54 — 59

NgRx Entity

NgRx Component Store

NgRx SignalStore (@ngrx/signals)

NgRx Router Store

NgRx Testing

NgRx Best Practices & Architecture Patterns



Complete topic content · All 16 sections per topic

Generated: May 17, 2026

Angular Learning Series · Continuation from Topic 53

Table of Contents

Topic 54 — NgRx Entity

Section 0–16: Difficulty · Explanation · Prior Concepts · Code · Before/After · Mistakes · Do's/Don'ts · Debugging · Real-World · Exercises · Takeaways · Question · Related · Checkpoint · Interview · Anti-Patterns · Mental Model

Topic 55 — NgRx Component Store

All 16 sections as above

Topic 56 — NgRx SignalStore (@ngrx/signals)

All 16 sections as above

Topic 57 — NgRx Router Store

All 16 sections as above

Topic 58 — NgRx Testing

All 16 sections as above

Topic 59 — NgRx Best Practices & Architecture Patterns

All 16 sections as above

Angular Learning Series

Topics 54 – 59

NgRx Entity · NgRx Component Store · NgRx SignalStore
NgRx Router Store · NgRx Testing · NgRx Best Practices

Generated: May 17, 2026

Complete topic content — all 16 sections per topic

TOPIC 54 — NgRx Entity

Angular & NgRx | Difficulty: Intermediate | Angular Relevance: High

Section 0 — Difficulty & Priority Rating

Difficulty: Intermediate — NgRx Entity introduces a new abstraction layer (the EntityAdapter) on top of the NgRx reducer pattern you already know, requiring you to learn a new API rather than new programming concepts.

Angular Relevance: High — Any Angular app managing lists of records (users, products, orders, messages) benefits directly from NgRx Entity, which eliminates hundreds of lines of hand-written CRUD reducer boilerplate while enforcing a consistent, performant data structure.

Section 1 — Explanation

The Filing Cabinet Analogy

Imagine you're managing paper records for a company. Without a system, you pile every employee file in a heap — to find one, you rifle through the whole stack. To update one record, you pull out the stack, find it, swap it, and put everything back. Slow and error-prone.

A professional filing cabinet solves this: every file gets a unique tab ID, files are stored alphabetically by ID, and you have a master index listing all IDs in order. To find, update, or delete any record, you go straight to the tab. Done.

NgRx Entity is that filing cabinet for your NgRx store.

Technically: NgRx Entity (@ngrx/entity) provides an EntityAdapter that manages a normalized data structure called an EntityState:

```
interface EntityState<T> {  
  ids: string[] | number[]; // ordered list of all entity IDs  
  entities: Record<string, T>; // dictionary: id → entity object  
}
```

Instead of writing your own find, filter, map, and splice logic in reducers, the adapter gives you ready-made CRUD operations. These are pure functions that return new state objects, compatible with NgRx's immutability requirements.

Quick Reference — EntityAdapter Methods

Method	What it does
addOne(entity, state)	Adds one entity; ignores if ID exists
addMany(entities, state)	Adds array of entities
setOne(entity, state)	Adds or replaces one entity

Method	What it does
setMany(entities, state)	Adds or replaces an array
setAll(entities, state)	Replaces entire collection
updateOne({ id, changes }, state)	Partial update on one entity
updateMany(updates, state)	Partial update on many
upsertOne(entity, state)	Add if missing, merge if exists
removeOne(id, state)	Removes one by ID
removeMany(ids, state)	Removes multiple by ID
removeAll(state)	Clears the collection
getInitialState(extra?)	Creates { ids: [], entities: {} }

Section 2 — Connection to Prior Concepts

Directly builds on:

- Topic 51 — NgRx Core Concepts: The reducer pattern, pure state functions, createReducer, on(), and createFeatureSelector/createSelector are all prerequisites. NgRx Entity simply provides helper functions that slot inside reducers you already know how to write.
- Topic 53 — NgRx Selectors In Depth: The adapter generates built-in selectors (selectAll, selectEntities, selectIds, selectTotal) that plug directly into the composed selector pattern from Topic 53.
- Topic 52 — NgRx Effects: Entity reducers are fed by actions dispatched from Effects — the HTTP-to-store pattern from Topic 52 is where entity data typically enters the store.
- Topic 14 — readonly & Immutability: The adapter's CRUD methods always return new state references, enforcing the immutability rule from Topic 14 automatically.

The single topic it extends most directly: Topic 51 — NgRx Core Concepts.

NgRx Entity is essentially a pre-written, battle-tested reducer library for collections. Everything in Topic 51 still applies — Entity just eliminates the hand-written CRUD code inside your on() handlers.

Bridging note: In Topic 51 you wrote reducers like `state.users.filter(u => u.id !== action.id)` for every CRUD operation. NgRx Entity replaces every one of those array operations with a single adapter method call, while the normalized `{ ids, entities }` structure makes selector lookups $O(1)$ instead of $O(n)$ array scans.

Section 3 — Code Example

Plain TS — What EntityAdapter does under the hood

```
// Without Entity – hand-written, error-prone
function removeUser(state: UsersState, id: string): UsersState {
```

```

return {
  ...state,
  users: state.users.filter(u => u.id !== id) // O(n) scan every time
};
}

// With Entity – adapter handles it
// adapter.removeOne(id, state) returns the same result, but:
// - uses the { ids, entities } normalized structure
// - is O(1) for lookup (dictionary access, not array scan)
// - is already tested by the NgRx team

```

Full Angular NgRx Entity Feature

```

// user.model.ts
export interface User {
  id: string;
  name: string;
  email: string;
  role: 'admin' | 'viewer';
}

// user.actions.ts
import { createActionGroup, emptyProps, props } from '@ngrx/store';
import { User } from './user.model';

export const UsersApiActions = createActionGroup({
  source: 'Users API',
  events: {
    'Users Loaded': props<{ users: User[] }>(),
    'User Added': props<{ user: User }>(),
    'User Updated': props<{ id: string; changes: Partial<User> }>(),
    'User Removed': props<{ id: string }>(),
    'Users Load Failed': props<{ error: string }>(),
  }
});

// user.reducer.ts
import { createEntityAdapter, EntityAdapter, EntityState } from '@ngrx/entity';
import { createFeature, createReducer, on } from '@ngrx/store';

export const userAdapter: EntityAdapter<User> = createEntityAdapter<User>({
  sortComparer: (a, b) => a.name.localeCompare(b.name),
});

```

```

export interface UsersState extends EntityState<User> {
  loading: boolean;
  error: string | null;
  selectedUserId: string | null;
}

const initialState: UsersState = userAdapter.getInitialState({
  loading: false,
  error: null,
  selectedUserId: null,
});

export const usersFeature = createFeature({
  name: 'users',
  reducer: createReducer(
    initialState,
    on(UsersApiActions.usersLoaded, (state, { users }) =>
      userAdapter.setAll(users, { ...state, loading: false, error: null })),
    on(UsersApiActions.userAdded, (state, { user }) =>
      userAdapter.addOne(user, state)),
    on(UsersApiActions.userUpdated, (state, { id, changes }) =>
      userAdapter.updateOne({ id, changes }, state)),
    on(UsersApiActions.userRemoved, (state, { id }) =>
      userAdapter.removeOne(id, state)),
  ),
});

// user.selectors.ts
const {
  selectAll: selectAllUsers,
  selectEntities: selectUserEntities,
  selectIds: selectUserIds,
  selectTotal: selectUserTotal,
} = userAdapter.getSelectors(selectUsersState);

export const selectSelectedUser = createSelector(
  selectUserEntities,
  selectSelectedUserId,

```



```
(entities, selectedId) => selectedId ? entities[selectedId] ?? null : null
);

export const selectAdminUsers = createSelector(
  selectAllUsers,
  (users) => users.filter(u => u.role === 'admin')
);
```

Section 4 — Before & After Refactor

Before — Hand-written array CRUD reducer

```
// BEFORE – writing all array operations manually
on(UsersApiActions.userUpdated, (state, { id, changes }) => ({
  ...state,
  // O(n) scan + new array + new object – 3 allocations per update
  users: state.users.map(u => u.id === id ? { ...u, ...changes } : u),
})),

on(UsersApiActions.userRemoved, (state, { id }) => ({
  ...state,
  users: state.users.filter(u => u.id !== id), // O(n) scan
})),
```

What breaks: $O(n)$ operations on every state change — with 10,000 users, every update scans the entire array. Selector by ID requires `users.find(u => u.id === id)` — another $O(n)$ scan. Duplicate IDs are not prevented. Sort order must be re-implemented manually.

After — NgRx Entity adapter

```
// AFTER – adapter handles all CRUD operations
on(UsersApiActions.userUpdated, (state, { id, changes }) =>
  userAdapter.updateOne({ id, changes }, state)
  // O(1) dictionary update
),

on(UsersApiActions.userRemoved, (state, { id }) =>
  userAdapter.removeOne(id, state)
  // O(1) delete
),
```

Why this refactor matters: Selector performance is $O(1)$ via dictionary lookup — critical with `OnPush` and large lists. Duplicate protection is built in. Sort order is maintained automatically by `sortComparer`. The adapter's methods are already tested by the NgRx team.

Section 5 — Common Mistakes & Misconceptions

Mistake 1 — Using `addOne` when you need `upsertOne` for API sync

What: Dispatching `addOne` when refreshing data from an API that returns records already in the store. `addOne` silently ignores records whose IDs are already present.

Why developers make it: `addOne` sounds like 'add this to the collection' — but its behaviour when the ID already exists is silent ignore, not update.

How to avoid: If the API can return records that might already be in the store, use `upsertOne` or `setAll`. Use `addOne` only for genuinely new records created by the user.

Angular-specific consequence: Your UI silently shows stale data. No error, no warning.

Mistake 2 — Calling `getSelectors()` without passing the feature selector

What: The no-argument form expects you to pass the feature state slice yourself at the `store.select()` call site, which is easy to forget.

How to avoid: Always pass the feature selector: `userAdapter.getSelectors(selectUsersState)`.

Angular-specific consequence: `selectAll` returns undefined or throws a runtime error.

Mistake 3 — Putting `sortComparer` logic in selectors instead of the adapter

What: Sorting in a selector creates a new array reference on every emission, breaking memoisation exactly as warned in Topic 53.

How to avoid: Define `sortComparer` in `createEntityAdapter()`. The adapter maintains sorted order in the `ids` array automatically.

Angular-specific consequence: The sorted selector emits a new array reference on every action, causing `OnPush` components to re-render unnecessarily.

Section 6 — Do's and Don'ts

DO	DON'T
Define <code>sortComparer</code> in <code>createEntityAdapter()</code> for ordered lists	Sort inside selectors — it returns a new array reference every emission
Use <code>upsertOne/upsertMany</code> when syncing from an API	Use <code>addOne</code> for API refreshes — it silently ignores existing records
Use <code>getSelectors(featureSelector)</code> with the feature selector argument	Call <code>getSelectors()</code> with no argument — it produces root-scoped selectors
Use <code>updateOne({ id, changes })</code> for partial updates	Spread the whole entity manually in the reducer
Extend <code>EntityState<T></code> with extra slices (loading, error, <code>selectedId</code>)	Force loading/error flags into the entity objects themselves
Register lazily in route providers with <code>provideState()</code>	Register entity feature state in <code>app.config.ts</code>

Section 7 — Debugging Tips

Bug 1 — selectAll returns empty array even after dispatching load success

Symptom: Redux DevTools shows state with entities populated but selectAll returns [].

Cause: getSelectors() called without a feature selector argument — reading from root state instead of feature slice.

Fix: Always use userAdapter.getSelectors(selectUsersState).

Bug 2 — Updating an entity does nothing visible in the component

Symptom: You dispatch updateOne, DevTools shows state updated, but the component doesn't re-render.

Cause: With OnPush, if strictStateImmutability is off and someone mutated state directly, the selector sees the same reference and skips emission.

Fix: Enable strictStateImmutability: true in NgRx runtime checks (development only).

Bug 3 — upsertOne is not merging — it's replacing the entire entity

Symptom: You call upsertOne with a partial entity and other fields disappear.

Cause: upsertOne adds the entity as-is if the ID doesn't exist. Since the ID is absent, it treats it as addOne and stores the partial object directly.

Fix: Use upsertOne for full entity objects only. For guaranteed partial updates on existing records, use updateOne({ id, changes }).

Section 8 — Real-World Applications

Scenario 1 — Product catalogue with real-time stock updates

An e-commerce Angular app receives WebSocket messages whenever a product's stock level changes. The store holds thousands of products. Each WebSocket update dispatches productUpdated with a partial change object. updateOne handles this as an O(1) dictionary update — no array scan regardless of catalogue size.

Scenario 2 — Inbox with read/unread state and selection

A messaging app thread list uses selectThreadEntities (dictionary) combined with selectSelectedThreadId for O(1) lookup of the selected thread. As the inbox grows to thousands of threads, selection lookup stays O(1). Without Entity, threads.find(t => t.id === selectedId) degrades linearly.

Section 9 — Practice Exercises

Beginner

Create an NgRx Entity feature for a Task entity with fields id, title, and completed. Set up the adapter, initial state, and a reducer that handles three actions: tasks loaded, task added, and

task removed. No component or effect — just the adapter, state interface, and reducer.

Intermediate

Extend the Task feature. Add a selectedTaskId slice to the state. Write a full selector file that exports: all tasks, completed tasks only, incomplete tasks only, the selected task entity (using dictionary lookup), and the count of incomplete tasks. Build a TasksListComponent using store.selectSignal() for all five selectors and ChangeDetectionStrategy.OnPush.

Advanced

Build a ContactsFeature that: (1) loads contacts from a fake HTTP service using an NgRx Effect; (2) supports optimistic delete — dispatch remove immediately, call the API, and if it fails dispatch a restore action that re-adds the contact using addOne; (3) uses sortComparer to keep contacts alphabetically sorted; (4) includes a search selector that filters selectAll by a searchTerm slice stored alongside entity state.

Section 10 — Key Takeaways

- EntityState<T> normalizes a collection into { ids: string[], entities: Record<string, T> } — O(1) CRUD operations
- createEntityAdapter<T>() produces all CRUD helper functions; sortComparer keeps ids array sorted automatically
- getInitialState(extras) merges the empty entity structure with your extra state slices
- getSelectors(featureSelector) produces four scoped selectors: selectAll, selectEntities, selectIds, selectTotal
- Adapter methods are pure functions returning new state — they satisfy NgRx immutability automatically

Operation	Array approach	Entity approach
Add entity	[...state.users, user]	adapter.addOne(user, state)
Update entity	state.users.map(...)	adapter.updateOne({ id, changes }, state)
Remove entity	state.users.filter(...)	adapter.removeOne(id, state)
Load all	{ ...state, users }	adapter.setAll(users, state)
Select by ID	users.find() — O(n)	entities[id] — O(1)
Prevent duplicates	Manual check required	addOne ignores duplicates automatically

One-Line Memory Aid: NgRx Entity is a pre-built filing cabinet — it normalizes your collection into a dictionary for O(1) lookups and gives you adapter methods that write immutable CRUD reducers for you.

Section 11 — Thought-Provoking Question + Answer

Question: NgRx Entity's selectAll selector returns a User[] array — a new array reference every time. Does this mean it always breaks memoisation and causes unnecessary OnPush re-renders?

Answer: No — and this is one of the subtlest points in NgRx Entity. selectAll is implemented as a createSelector internally. Like all NgRx selectors, it uses referential equality checking on its inputs, not its output. The selector only recomputes when state.ids or state.entities changes reference.

An action that doesn't touch the users feature slice leaves input references unchanged — selector short-circuits — same User[] reference returned — OnPush does NOT re-render.

An action that changes entity data produces new ids or entities reference — selector recomputes — new User[] reference — OnPush component re-renders correctly.

Practical rule: Trust selectAll memoisation. Only audit derived selectors that filter, sort, or reshape the result — those are where new reference creation silently breaks OnPush.

Section 12 — Related Topics to Learn Next

ES6/JS Concepts

- Object property access patterns — why entities[id] is O(1) while array.find() is O(n)
- Spread operator with nested objects — how updateOne produces new entity references immutably

TypeScript Concepts

- Mapped types (Record<K,V>) — EntityState.entities is typed as Record<string, T>
- Generics with constraints — createEntityAdapter<T>() uses generics

Angular-Specific Concepts

- Topic 55 — NgRx Component Store: After Entity for global collections, Component Store handles local entity-like state
- Topic 58 — NgRx Testing: Testing Entity reducers requires adapter.getInitialState() for test state setup
- Topic 59 — NgRx Best Practices: Where Entity fits in the broader NgRx architecture

Section 13 — Study Plan Checkpoint

Topics this directly depends on: Topic 13 (TypeScript Interfaces), Topic 14 (Immutability), Topic 51 (NgRx Core), Topic 52 (Effects), Topic 53 (Selectors).

Topics this unlocks: Topic 55 (Component Store), Topic 58 (NgRx Testing), Topic 59 (NgRx Best Practices).

Production readiness: NgRx Entity is production-ready and actively maintained — it is the recommended approach for any collection of identifiable records in an NgRx store.

Section 14 — Common Interview Questions

Q1 (Mid): What problem does NgRx Entity solve and how does its data structure improve performance over a plain array?

Strong answer covers: Arrays require $O(n)$ scan for find/update/delete; Entity normalizes to { ids, entities } enabling $O(1)$ dictionary lookup. Eliminates hand-written CRUD boilerplate. The ids array preserves ordered selection while entities dictionary enables fast keyed access. sortComparer maintains order automatically.

Weak answers miss: The performance argument ($O(1)$ vs $O(n)$).

Q2 (Mid): What's the difference between addOne, setOne, and upsertOne?

Strong answer: addOne adds only if ID absent (silently ignores if present). setOne unconditionally adds or replaces the entire entity. upsertOne adds if absent, merges if present.

Weak answers miss: The distinction between setOne (full replace) and upsertOne (merge).

Q3 (Senior): A developer uses sortComparer in their EntityAdapter but also sorts again inside a selector. Consequences?

Strong answer: sortComparer means selectAll already returns sorted entities. Sorting in selector projector always returns new array reference — memoisation fails — selector emits on every action touching feature slice — OnPush components re-render unnecessarily.

Weak answers miss: The memoisation consequence and OnPush re-render behaviour.

Section 15 — Anti-Patterns to Avoid in Production

Anti-Pattern 1 — Sorting inside a selector on top of selectAll

```
// WRONG
export const selectSortedUsers = createSelector(
  selectAllUsers,
  (users) => [...users].sort((a, b) => a.name.localeCompare(b.name))
);

// CORRECT
export const userAdapter = createEntityAdapter<User>({
  sortComparer: (a, b) => a.name.localeCompare(b.name),
});
// selectAll already returns sorted users
```

Why it seems fine: Works correctly — users appear sorted. Why it breaks: New [] reference on every emission breaks OnPush.

Anti-Pattern 2 — Registering entity feature state in app.config.ts

Why it seems fine: Simple, all state immediately available everywhere. What breaks: All entity state eagerly downloaded for every user on every page — defeats lazy loading, increases bundle

size.

Correct: Use `provideState('users', usersReducer)` in route providers.

Anti-Pattern 3 — Creating `store.select()` inside a component method for per-entity lookups

Why it seems fine: Looks like a convenient per-item lookup. What breaks: A new Observable subscription created and destroyed on every change detection cycle for every item — thousands of subscriptions per second in large lists.

Correct: Pass IDs to child components; each child selects its own entity once.

Section 16 — Mental Model Summary

Core concept: NgRx Entity normalizes a collection of records into a `{ ids[], entities{} }` dictionary structure and provides an `EntityAdapter` with $O(1)$ CRUD methods that write immutable reducers for you.

Rule	Detail
1. <code>getSelectors(featureSelector)</code>	Always pass feature selector — no-argument form reads from root state
2. <code>sortComparer</code> in adapter	Never sort in selectors — new array refs break memoisation
3. <code>upsertOne</code> vs <code>updateOne</code>	<code>upsertOne</code> for API sync; <code>updateOne</code> for partial updates on known records
4. Extend <code>EntityState<T></code>	Add extra slices (loading, error, <code>selectedId</code>) in <code>getInitialState()</code>
5. Lazy registration	<code>provideState()</code> in route providers — never in <code>app.config.ts</code>
6. Always <code>OnPush</code>	$O(1)$ dictionary lookup makes selector memoisation most effective

Most important rule: Never sort inside a selector on top of `selectAll` — define `sortComparer` in the adapter so order is maintained at write time, not recomputed at read time.

TOPIC 55 — NgRx Component Store

Angular & NgRx | Difficulty: Intermediate | Angular Relevance: High

Section 0 — Difficulty & Priority Rating

Difficulty: Intermediate — Component Store introduces a new mental model (local vs global state) and a reactive updater/effect pattern, but builds directly on RxJS and NgRx concepts you already know.

Angular Relevance: High — Component Store solves the gap between component state too complex for signals but too local for the global NgRx store — the right tool for data tables, wizards, and sidepanels.

Section 1 — Explanation

The Whiteboard Analogy

Imagine your company has two types of information storage: The central database (NgRx global store) holds company-wide records that every department needs. A personal whiteboard in each meeting room is used only during a meeting. When the meeting ends, the whiteboard gets wiped. Nobody else in the building cares what's on it.

NgRx Component Store is the whiteboard.

Technically: `@ngrx/component-store` provides a `ComponentStore<StateType>` base class. You extend it and get: `setState()`, `updater()`, `select()`, `effect()`, `patchState()`, and automatic subscription cleanup when the component is destroyed.

	NgRx Global Store	NgRx Component Store
Scope	Entire application	One component + children
Lifetime	App session	Component lifetime
State setup	<code>createFeature + provideState()</code>	Extend <code>ComponentStore<T></code>
Write mechanism	Dispatch actions → reducers	<code>updater()</code> or <code>patchState()</code>
Side effects	<code>createEffect()</code> in Effects class	<code>effect()</code> method
DevTools	Full time-travel debugging	Limited
Boilerplate	High	Medium

Section 2 — Connection to Prior Concepts

- Topic 51 — NgRx Core Concepts: The global store's one-way data flow is mirrored locally in Component Store.

- Topic 52 — NgRx Effects: The effect() method works identically — same switchMap/concatMap/exhaustMap rules apply.
- Topic 20 & 21 — RxJS Subscription Management: Component Store handles cleanup automatically.
- Topic 44 — State Management Patterns: The BehaviorSubject store pattern is exactly what Component Store formalizes.
- Topic 27 — Services & DI: Component Store is provided in the component's providers: [] array.

The single topic it extends most directly: Topic 44 — State Management Patterns. Component Store formalizes the hand-rolled store service with additional reactive discipline.

Section 3 — Code Example

```
// data-table.store.ts
@Injectable()
export class DataTableStore extends ComponentStore<DataTableState> {
  constructor(private userService: UserService) {
    super(initialState); // Required – initializes internal BehaviorSubject
  }

  // SELECTORS – memoised Observables
  readonly users$ = this.select(state => state.users);
  readonly filter$ = this.select(state => state.filter);
  readonly filteredUsers$ = this.select(
    this.users$, this.filter$,
    (users, filter) => filter
    ? users.filter(u => u.name.toLowerCase().includes(filter.toLowerCase()))
    : users
  );
  // View model selector – single subscription for template
  readonly vm$ = this.select(
    this.filteredUsers$, this.loading$, this.error$,
    (users, loading, error) => ({ users, loading, error })
  );

  // UPDATERS – pure functions returning new state
  readonly setFilter = this.updater((state, filter: string) => ({
    ...state, filter, currentPage: 1,
  }));

  // EFFECTS – side effects with tapResponse for error safety
```

```

readonly loadUsers = this.effect((trigger$: Observable<void>) =>
trigger$.pipe(
switchMap(() => {
this.patchState({ loading: true, error: null });
return this.userService.getUsers().pipe(
tapResponse(
(users) => this.patchState({ users, loading: false }),
(error: Error) => this.patchState({ error: error.message, loading: false })
)
);
})
);
}
// data-table.component.ts
@Component({
changeDetection: ChangeDetectionStrategy.OnPush,
providers: [DataTableStore], // Component-scoped lifetime
template: `
@if (vm$ | async; as vm) {
@if (vm.loading) { <p>Loading...</p> }
@for (user of vm.users; track user.id) {
<div>{{ user.name }}</div>
}
}
`
})
export class DataTableComponent implements OnInit {
private store = inject(DataTableStore);
vm$ = this.store.vm$;
ngOnInit() { this.store.loadUsers(); }
onFilter(q: string) { this.store.setFilter(q); }
}

```

Section 4 — Before & After Refactor

Before — Complex local state managed inside the component

State is scattered as individual class properties. `filteredUsers` is a getter that recomputes on every CD cycle with no memoisation. Business logic (HTTP calls, filtering) lives inside the component — untestable in isolation. Manual `Subject` + `takeUntil` for cleanup.

After — Component Store

The store is independently testable. `select()` is memoised — `filteredUsers$` only recomputes when inputs change. Cleanup is automatic. Child components can inject the same store instance via the injector hierarchy.

Section 5 — Common Mistakes & Misconceptions

Mistake 1 — Providing Component Store in root instead of the component

Using `providedIn: 'root'` creates a singleton that persists for the entire app session. State accumulates across every component instance that has ever used it and never resets when the component is destroyed.

Mistake 2 — Mutating state directly inside `updater()`

Calling `state.users.push(user)` and returning the same state object means `select()` never detects a change. Always return a new state object: `{ ...state, users: [...state.users, user] }`.

Mistake 3 — Not using `tapResponse` inside `effect()` and letting errors crash the effect

An unhandled error in `effect()` permanently kills the outer stream. Calling `loadUsers()` again silently does nothing. Always use `tapResponse(successFn, errorFn)` — identical to the Topic 52 `catchError` placement rule.

Section 6 — Do's and Don'ts

DO	DON'T
Provide in component's providers: <code>[]</code>	Use <code>providedIn: 'root'</code> — creates a persistent singleton
Use <code>tapResponse(successFn, errorFn)</code> in every <code>effect()</code>	Use plain <code>tap()</code> — unhandled error permanently kills the effect
Return new state object from <code>updater()</code>	Mutate state argument and return it — breaks <code>select()</code> memoisation
Use <code>vm\$</code> view model selector pattern	Subscribe to each state slice separately
Use <code>patchState()</code> for simple flag flips	Always use <code>updater()</code> for everything
Define effects as class properties	Create effects dynamically inside methods — leaks subscriptions

Section 7 — Debugging Tips

Bug 1 — Store state resets every time the component re-renders

Cause: `DataTableStore` is being instantiated directly (`new DataTableStore()`) instead of injected via providers. Fix: Use providers: `[DataTableStore]` then `inject(DataTableStore)`.

Bug 2 — Child component cannot inject the parent's Component Store

Cause: Content-projected components use the projecting parent's injector, not the host component's. Fix: Use providers: [] (not viewProviders) in the parent.

Bug 3 — effect() is called but nothing happens

Cause: super(initialState) was omitted from the constructor. The internal BehaviorSubject is uninitialized and emissions are silently dropped. Always call super(initialState) as the first line.

Section 8 — Real-World Applications

Scenario 1 — Multi-step form wizard

A checkout wizard with five steps uses CheckoutWizardStore provided in the wizard component. State includes currentStep, stepData for each step, and submitting flag. When the wizard component is destroyed, the entire wizard state is automatically cleaned up — no lingering order data in the global store.

Scenario 2 — Multiple independent activity feed widgets

A dashboard shows multiple activity feed widgets. The same ActivityFeedStore class powers all of them simultaneously with completely separate state. Three ActivityFeedComponent instances on the same dashboard each get their own store instance — completely independent HTTP calls and page counters. Impossible to achieve cleanly with a single global NgRx store slice without complex keying.

Section 9 — Practice Exercises

Beginner

Create a CounterStore extending ComponentStore<{ count: number }> with initial count 0. Add three updaters: increment, decrement, and reset. Add one selector: count\$. Build a CounterComponent that provides the store and uses the three updaters for three buttons.

Intermediate

Build a SearchStore managing { query, results, loading, error }. Implement setQuery updater, filteredResults\$ selector, and a search effect using switchMap and tapResponse. Connect to a SearchComponent with a debounced input. Use the view model selector pattern.

Advanced

Build a DataGridStore for a paginated, sortable, filterable data grid. Include items, loading, error, sortField, sortDirection, filter, currentPage, totalPages. Implement loadPage effect with switchMap. Include vm\$ selector. Child row components inject the same store instance to dispatch selectRow directly. Verify with OnPush on both components.

Section 10 — Key Takeaways

- ComponentStore<T> is self-contained reactive state scoped to one component's lifetime

- `updater((state, payload) => newState)` is a mini-reducer — always return a new state object
- `select(projectorFn)` creates a memoised Observable — same referential equality rules as `createSelector`
- `effect(trigger$ => Observable)` is a local NgRx Effect — same operator rules apply
- `tapResponse(successFn, errorFn)` handles responses safely — always use inside effects
- `patchState({ key: value })` is simpler alternative to `updater()` for partial updates

One-Line Memory Aid: Component Store is NgRx for one component — same `updater/selector/effect` patterns, but scoped to a single component's lifetime with no global boilerplate required.

Section 11 — Thought-Provoking Question + Answer

Question: If Component Store and a hand-rolled BehaviorSubject store service both solve local state management, what does Component Store actually add?

Answer: Component Store adds: (1) `updater()` enforces pure-function mutation discipline — type system prevents returning same reference. (2) `select()` provides automatic memoisation via `distinctUntilChanged` internally. (3) `effect()` manages inner Observable lifecycle automatically. (4) Automatic cleanup — no manual `takeUntilDestroyed` wiring needed.

When the simpler Topic 44 BehaviorSubject service is the right choice: one or two boolean flags, need for fine-grained subscription timing control, or team not already using NgRx.

Practical rule: Use Component Store when the component has three or more interdependent state slices or at least one async side effect.

Section 12 — Related Topics to Learn Next

- ES6: Class inheritance and `super()` — ComponentStore requires `super(initialState)` to initialize internal BehaviorSubject
- TypeScript: Generic class constraints — `ComponentStore<T>` is generic; state type constrains all methods
- Topic 56 — NgRx SignalStore: The Signal-native evolution of Component Store
- Topic 58 — NgRx Testing: Testing Component Store involves providing it in TestBed with a mock service
- Topic 59 — NgRx Best Practices: Where to draw the line between Component Store and global NgRx

Section 13 — Study Plan Checkpoint

Topics this depends on: Topic 20 (RxJS Subscriptions), Topic 21 (`takeUntilDestroyed`), Topic 27 (DI), Topic 44 (State Management Patterns), Topic 46 (RxJS Operators), Topic 52 (Effects), Topic 53 (Selectors).

Topics this unlocks: Topic 56 (NgRx SignalStore), Topic 58 (NgRx Testing), Topic 59 (NgRx Best Practices).

Production readiness: Component Store is production-ready; in new Angular 17+ projects, evaluate NgRx SignalStore first for Signal-native integration.

Section 14 — Common Interview Questions

Q1 (Mid): What is NgRx Component Store and how does it differ from the global NgRx Store?

Strong answer: Global Store — app-wide lifetime, actions/reducers/effects wiring, full DevTools. Component Store — component lifetime, no action ceremony, simpler API. Choose global for shared/persistent data; Component Store for UI state local to one feature.

Weak answers miss: The lifetime scoping argument.

Q2 (Mid): What is tapResponse and why use it instead of plain tap inside an effect()?

Strong answer: tapResponse catches errors inside the inner Observable. Plain tap with no error handling — an HTTP error propagates to the outer effect stream and permanently kills it. After the outer stream errors, calling the effect again silently does nothing.

Weak answers miss: That the effect is permanently killed (not just skipped).

Q3 (Senior): A developer provides Component Store in providers: [] at the component level. A child and grandchild both inject it. Explain what each gets — and the one scenario where this breaks.

Strong answer: Child and grandchild find the store in the parent's injector — same instance. Breaks with content projection: a component projected via ng-content uses its declaring parent's injector, not the host component's — NullInjectorError if host is the only provider.

Weak answers miss: The content projection edge case.

Section 15 — Anti-Patterns to Avoid in Production

Anti-Pattern 1 — Providing Component Store in root

@Injectable({ providedIn: 'root' }) creates a singleton. Opening a data table modal twice shows stale state from the first open. State never resets. Always use component providers: [DataTableStore].

Anti-Pattern 2 — Creating a new effect() call inside a method

Every call to loadData() creates a brand new effect — a new internal Subject, a new subscription chain. These accumulate for the component's lifetime. A component that calls loadData() on every row click leaks a new subscription chain on every click. Define effects as class properties, called once.

Anti-Pattern 3 — Sharing Component Store state between unrelated components via a parent provider

Providing a store at AppComponent level for unrelated features creates an informal global store with none of DevTools support, action logging, or immutability enforcement. State persists for the entire app session. Correct: if state is needed by unrelated features, it belongs in the global NgRx store.

Section 16 — Mental Model Summary

Core concept: NgRx Component Store is a self-contained, component-scoped reactive state manager — same updater/selector/effect discipline as NgRx, but with no global registration, no actions, and automatic cleanup when the component is destroyed.

Rule	Detail
1. providers: []	Always provide in component — never providedIn: 'root'
2. super(initialState)	Always call in constructor — omitting leaves BehaviorSubject uninitialized
3. tapResponse always	Unhandled errors permanently kill the effect silently
4. New state from updater()	Never mutate the state argument — return new object with spread
5. Effects as class properties	Never create effects dynamically inside methods — leaks subscriptions
6. Use Component Store when...	3+ interdependent slices or any async side effect

Most important rule: Always use tapResponse inside every effect() — one unhandled HTTP error permanently kills the effect for the component's lifetime with no warning and no recovery.

TOPIC 56 — NgRx SignalStore (@ngrx/signals)

Angular & NgRx | Difficulty: Intermediate | Angular Relevance: High

Section 0 — Difficulty & Priority Rating

Difficulty: Intermediate — SignalStore introduces a new composition API (`withState`, `withComputed`, `withMethods`, `withHooks`) that feels different from both Component Store and global NgRx, but the underlying concepts are ones you already know from Signals (Topic 23) and Component Store (Topic 55).

Angular Relevance: High — SignalStore is the NgRx team's Signal-native answer to Component Store, designed specifically for Angular 17+ apps built on Signals and `ChangeDetectionStrategy.OnPush`; it eliminates the RxJS-Observable layer entirely for state management in components that have committed to the Signal model.

Section 1 — Explanation

The LEGO Block Analogy

Imagine building with LEGO. The old approach (Component Store) gives you a pre-shaped base plate — a fixed structure you must build on top of, following its dimensions.

SignalStore gives you individual LEGO bricks that snap together in any combination: `withState(...)` — the brick that holds raw state values. `withComputed(...)` — the brick that derives new values from state. `withMethods(...)` — the brick that adds updaters and effects. `withHooks(...)` — the brick that taps into the store's lifecycle.

Each brick is independently composable. You snap together only the bricks you need. The store is the result of all the bricks snapped together — a plain Angular injectable with Signal-based properties.

Quick Reference — SignalStore vs Component Store vs Global NgRx

	Global NgRx	Component Store	SignalStore
API style	Class + decorators	Class extension	Function composition
State type	Plain objects	Plain objects	Angular Signals
Read state	<code>select()</code> → Observable	<code>select()</code> → Observable	<code>store.slice</code> → Signal
Write state	Dispatch action	<code>updater()/patchState()</code>	<code>patchState(store, {...})</code>
Derived state	<code>createSelector()</code>	<code>select(projector)</code>	<code>withComputed(() => computed())</code>
Template binding	async pipe or <code>selectSignal</code>	async pipe	<code>store.count()</code> directly
Lifecycle hooks	n/a	n/a	<code>withHooks({onInit, onDestroy})</code>

Section 2 — Connection to Prior Concepts

- Topic 23 — Angular Signals: SignalStore state slices are Angular Signals — `store.count()` is a Signal call, `withComputed` uses `computed()`.
- Topic 55 — NgRx Component Store: SignalStore solves the same problem but replaces Observable/RxJS API with Signals. `updater()` becomes `patchState()`; `select()` becomes `withComputed()`; `effect()` becomes `withMethods()` + `rxMethod()`.
- Topic 24 — OnPush Change Detection: SignalStore pairs perfectly — Signals notify Angular's runtime directly without `Zone.js`.
- Topic 53 — NgRx Selectors In Depth: `withComputed()` produces `computed()` signals — same memoisation rules apply.

The single topic it extends most directly: Topic 55 — NgRx Component Store. SignalStore is the Signal-native evolution with identical architectural concepts but Signals replacing Observables.

Section 3 — Code Example

```
// user.store.ts
interface UserState {
  users: User[];
  selectedUserId: string | null;
  loading: boolean;
  error: string | null;
  filter: string;
}

export const UserStore = signalStore(
  // BRICK 1: Raw state – each key becomes a Signal on the store
  withState<UserState>({ users: [], selectedUserId: null,
    loading: false, error: null, filter: '' }),

  // BRICK 2: Computed Signals
  withComputed((store) => ({
    filteredUsers: computed(() => {
      const filter = store.filter().toLowerCase();
      return filter
        ? store.users().filter(u => u.name.toLowerCase().includes(filter))
        : store.users();
    }),
    totalUsers: computed(() => store.users().length),
    hasError: computed(() => store.error() !== null),
  })),
```

```

// BRICK 3: Methods (mutations + effects)
withMethods((store, userService = inject(UserService)) => ({
  setFilter(filter: string) {
    patchState(store, { filter, selectedUserId: null });
  },
  loadUsers: rxMethod<void>(
    pipe(
      tap(() => patchState(store, { loading: true, error: null })),
      switchMap(() =>
        userService.getAll().pipe(
          tapResponse({
            next: (users) => patchState(store, { users, loading: false }),
            error: (err: Error) =>
              patchState(store, { error: err.message, loading: false }),
          })
        )
      )
    )
  ),
})),

// BRICK 4: Lifecycle hooks
withHooks({
  onInit(store) { store.loadUsers(); },
})
);

// users.component.ts
@Component({
  changeDetection: ChangeDetectionStrategy.OnPush,
  providers: [UserStore], // Component-scoped
  template: `
    @if (store.loading()) { <p>Loading...</p> }
    <input [value]='store.filter()'
      (input)='store.setFilter($event.target.value)' />
    @for (user of store.filteredUsers(); track user.id) {
      <div>{{ user.name }}</div>
    }
  `
})

export class UsersComponent {

```

```

store = inject(UserStore);
// No ngOnInit – withHooks.onInit fires automatically
// No async pipe – Signals read directly: store.loading()
// No subscription management – SignalStore handles cleanup
}

```

Section 4 — Before & After Refactor

Before — Component Store with Observable-based template binding

Required: async pipe, an ngOnInit to trigger loading, Observable mental model throughout the template, and RxJS knowledge to understand the effect chain. Template needed @if (vm\$ | async; as vm) wrapper.

After — SignalStore with Signal-based template binding

No async pipe means no null-check wrapper cluttering the template. withHooks.onInit replaces ngOnInit for triggering initial loads — the component class can be empty. Signal reads are synchronous. OnPush integrates naturally — Signals notify Angular directly without Zone.js.

Section 5 — Common Mistakes & Misconceptions

Mistake 1 — Reading a computed Signal inside withComputed before it's defined

JavaScript object literals are evaluated in definition order. If vm references store.filteredUsers and filteredUsers is defined below vm in the same withComputed() object, store.filteredUsers is undefined at vm's evaluation time. Fix: define in dependency order, or split into two withComputed() calls.

Mistake 2 — Using patchState outside of withMethods

Calling patchState(this.store, { filter }) directly from a component bypasses all withMethods encapsulation. At team scale, state mutations scattered across components make the store's state unpredictable. Always define a method in withMethods() and call that from the component.

Mistake 3 — Treating rxMethod like a regular function

rxMethod's power is accepting a Signal or Observable to create a reactive pipeline. Calling store.search(store.query) passes the Signal itself (not its value) — rxMethod auto-subscribes and reacts to every Signal emission, applying debounceTime and switchMap automatically.

Section 6 — Do's and Don'ts

DO	DON'T
Define computed Signals in dependency order inside withComputed()	Forward-reference a computed Signal defined later in the same object
Put all state mutations in withMethods()	Call patchState(store, ...) directly from a component

DO	DON'T
Use <code>withHooks({ onInit })</code> for initial data loading	Trigger initial loads in <code>ngOnInit</code> when store can self-initialize
Use <code>rxMethod()</code> for Observable chains and HTTP calls	Use plain <code>async/await</code> for complex Observable chains
Provide store in providers: <code>[]</code> for component scope	Mix scoping — component-local store provided at root
Use <code>tapResponse</code> inside <code>rxMethod</code> pipes	Omit error handling — first HTTP error permanently kills the method

Section 7 — Debugging Tips

Bug 1 — `store.someMethod` is not a function at runtime

Cause: `withMethods()` brick was accidentally omitted, or store was instantiated directly with new `UserStore()` bypassing DI. Fix: Verify `signalStore()` includes `withMethods(...)` and always inject via Angular DI.

Bug 2 — `computed()` in `withComputed` returns stale data after `patchState`

Cause: The `computed()` projector is reading a non-Signal value — a local variable captured by closure at creation time. Fix: Every value read inside `computed()` must be a Signal call with `()` to establish reactive dependency.

Bug 3 — `rxMethod` does nothing after the first call

Cause: An unhandled error in the `rxMethod` pipe killed the outer stream. First call works; subsequent calls silently do nothing. Fix: Always use `tapResponse` from `@ngrx/operators` inside `rxMethod` — it catches errors inside the inner Observable so the outer stream stays alive.

Section 8 — Real-World Applications

Scenario 1 — Shopping cart with persisted state and computed totals

A cart feature uses `providedIn: 'root'` `SignalStore` persisting across navigation. Computed signals derive `itemCount`, `subtotal`, `tax`, and `total` — each memoised. The `tax` computed reads `store.subtotal()` (another computed Signal) — chained memoisation. Any component in the app reads `cart.total()` directly with no `async` pipe.

Scenario 2 — Reactive search with debounce using `rxMethod` + Signal binding

A search store defines `setQuery(query)` and a search `rxMethod`. In `withHooks.onInit`, `store.search(store.query)` passes the Signal itself to `rxMethod`. Every time `store.query` changes, the `rxMethod` pipe re-runs — `debounceTime(300)`, `distinctUntilChanged()`, and `switchMap` applied automatically without manual subscription management.

Section 9 — Practice Exercises

Beginner

Create a `TodoStore` using `signalStore()` with state `{ todos: Todo[], filter: 'all'|'active'|'completed' }`. Add `withComputed` for `filteredTodos` and `completedCount`. Add `withMethods` for `addTodo(title)`, `toggleTodo(id)`, and `setFilter(filter)`. Provide in `TodoListComponent` and render with no `async pipe`.

Intermediate

Extend `TodoStore` to persist todos to `localStorage`. Add `withHooks` that loads todos from `localStorage` in `onInit` and saves in `onDestroy`. Add a `syncWithServer` method using `rxMethod<void>` that fetches todos from a fake service and merges them. Handle loading and error state with `tapResponse`.

Advanced

Build a `PaginatedTableStore` with state including `items`, `loading`, `error`, `currentPage`, `pageSize`, `sortField`, `sortDirection`, `totalItems`. Implement: (1) `loadPage rxMethod` with `switchMap`; (2) computed Signals for `totalPages`, `hasNextPage`, `hasPrevPage`, and a `vm` Signal; (3) `withHooks.onInit` loading first page; (4) `selectRow` method; (5) child `TableRowComponent` injecting same store instance. Verify with `OnPush` on both components.

Section 10 — Key Takeaways

- `signalStore()` is a factory composing feature bricks and returning an Angular injectable class
- State slices in `withState()` become Signals on the store — read with `store.slice()`, not `store.slice`
- `withComputed()` defines computed() Signals — memoised, reactive, dependency-tracked automatically
- `withMethods()` defines the store's public API — all `patchState()` calls live here
- `rxMethod<T>()` bridges RxJS into the Signal world — accepts plain value, Signal, or Observable
- `withHooks({ onInit, onDestroy })` replaces `ngOnInit/ngOnDestroy` for store-level lifecycle

One-Line Memory Aid: `SignalStore` is `Component Store` rebuilt with LEGO bricks — compose `withState`, `withComputed`, `withMethods`, and `withHooks` into an injectable whose state slices are Signals you read directly in templates without `async pipe`.

Section 11 — Thought-Provoking Question + Answer

Question: `SignalStore` state slices are Signals, and `withComputed()` uses `computed()` — both already provided by Angular natively. What does `SignalStore` actually add beyond wrapping Angular's own Signal primitives?

Answer: `SignalStore`'s value is architectural, not primitive. (1) Encapsulation enforcement — raw `signal()` is publicly writable; `SignalStore` state Signals are read-only externally. The only mutation path is through `withMethods()`. (2) Composable feature pattern — `withX()` bricks give teams a consistent architectural grammar. (3) RxJS interop — `rxMethod()` handles the Signal/Observable

boundary cleanly. (4) Lifecycle integration — withHooks integrates store initialization into Angular's DI lifecycle.

Practical rule: Use `raw signal()` for 1-2 independent values. Use `SignalStore` for 3+ interdependent slices, any HTTP effect, or state that children need to share.

Section 12 — Related Topics to Learn Next

- ES6: Function composition — `signalStore()` is built on function composition; each `withX()` call transforms the store type
- TypeScript: Intersection types — the store's type is the intersection of all `withX()` return types
- Topic 57 — NgRx Router Store: Router Store connects Angular Router state to NgRx
- Topic 58 — NgRx Testing: Testing `SignalStore` involves providing it in `TestBed` with mock services
- Topic 59 — NgRx Best Practices: The architectural decision of when to use `SignalStore` vs `Component Store` vs global NgRx

Section 13 — Study Plan Checkpoint

Topics this depends on: Topic 23 (Angular Signals), Topic 24 (OnPush), Topic 27 (DI), Topic 46 (RxJS Operators), Topic 55 (Component Store).

Topics this unlocks: Topic 57 (Router Store), Topic 58 (NgRx Testing), Topic 59 (NgRx Best Practices).

Production readiness: NgRx `SignalStore` reached stable release in NgRx 18 — production-ready for Angular 17+ Signal-first applications.

Section 14 — Common Interview Questions

Q1 (Mid): What is NgRx `SignalStore` and how does it differ from NgRx `Component Store`?

Strong answer: `Component Store` — class extension, Observable-based, requires async pipe. `SignalStore` — function composition, Signal-based, no async pipe. `SignalStore` integrates more naturally with Angular 17+ Signal-based reactivity. For new projects on Angular 17+: `SignalStore`. Existing `Component Store` projects: no urgent migration needed.

Q2 (Senior): Explain `rxMethod` in NgRx `SignalStore`. What inputs does it accept and what problem does a plain async method not solve?

Strong answer: `rxMethod<T>()` accepts a plain value, a Signal, or an Observable. When passed a Signal, it auto-subscribes and re-runs the pipe on every Signal emission. The pipe can use `debounceTime`, `switchMap`, `concatMap` — full RxJS operator semantics including request cancellation. A plain async method cannot cancel in-flight requests, debounce, or reactively bind to a Signal.

Q3 (Senior): A computed Signal in your SignalStore is not updating when state changes. Walk through debugging.

Strong answer: (1) Confirm computed reads Signal values with () calls — reading plain variables creates no reactive dependency. (2) Confirm patchState returns new references. (3) Check definition order in withComputed() — computed value cannot reference another defined later in same object. (4) Check if split across two withComputed() calls correctly. Angular DevTools Signal graph shows reactive dependency chain.

Section 15 — Anti-Patterns to Avoid in Production

Anti-Pattern 1 — Returning new object literals from withComputed projectors unnecessarily

computed(() => ({ name: store.selectedUser()?.name, count: store.users().length }))) creates a new object literal on every Signal emission — new reference — every component reading it detects a change and re-renders even with OnPush. Correct: return primitive values from individual computed Signals.

Anti-Pattern 2 — Calling patchState directly from the component

patchState is a public export — TypeScript permits it. At team scale, state mutations scattered across components make the store unpredictable. No single place to add validation, side effects, or logging. Always define named methods in withMethods() and call those from components.

Anti-Pattern 3 — Defining rxMethod inside withHooks instead of withMethods

rxMethod created inside onInit is a local variable — cannot be called again from outside. If you need to refresh data later, there's no reference. Additionally, subscription cleanup is tied to local variable scope, not store lifetime. Always define rxMethod in withMethods(), call it from withHooks.onInit.

Section 16 — Mental Model Summary

Core concept: NgRx SignalStore composes feature bricks (withState, withComputed, withMethods, withHooks) into an Angular injectable whose state slices are read-only Signals consumed directly in templates without async pipe, subscriptions, or Observable ceremony.

Rule	Detail
1. All mutations in withMethods()	Never call patchState from a component — use named methods
2. Dependency order	Define computed Signals before referencing them in same withComputed()
3. tapResponse always	Unhandled errors in rxMethod permanently kill the method silently
4. Signal not value to rxMethod	store.search(store.query) not store.search(store.query())
5. withHooks for loading	Keeps component class clean of lifecycle methods
6. No new literals in computed()	New object/array references break memoisation and OnPush

Most important rule: All `patchState()` calls belong inside `withMethods()` — exposing state mutation through named methods is what makes a `SignalStore` auditable, testable, and maintainable at team scale.

TOPIC 57 — NgRx Router Store

Angular & NgRx | Difficulty: Intermediate | Angular Relevance: High

Section 0 — Difficulty & Priority Rating

Difficulty: Intermediate — Router Store introduces one new concept (connecting the Angular Router to NgRx) but relies entirely on patterns you already know: selectors, effects, and the global store.

Angular Relevance: High — In any NgRx application where components read route params, query params, or the current URL to drive data loading, Router Store eliminates direct `ActivatedRoute` injection and makes the URL a first-class, selectable, testable slice of NgRx state.

Section 1 — Explanation

The GPS Tracker Analogy

Imagine a logistics company. Without a central system, each warehouse manager calls the driver directly to ask 'where are you?' — direct, coupled, hard to monitor or replay.

With a central dispatch system, every GPS update is logged to a shared board. Any manager reads the board. Operations can replay the route history. No one calls the driver directly.

NgRx Router Store is the dispatch system for your URL. Without it: components inject `ActivatedRoute` directly. With it: every navigation event is dispatched as an NgRx action and the current router state is stored as a slice of NgRx state.

Action	When it fires
<code>routerRequestAction</code>	Navigation starts (before guards run)
<code>routerNavigationAction</code>	Guards passed — navigation is committing
<code>routerNavigatedAction</code>	Navigation fully complete — component activated
<code>routerCancelAction</code>	A guard cancelled the navigation
<code>routerErrorAction</code>	Navigation threw an error

Section 2 — Connection to Prior Concepts

- Topic 38 — Angular Router: Router Store wraps the Router's event system — route params, query params, and navigation events are what Router Store exposes to NgRx.
- Topic 51 — NgRx Core Concepts: Router state is stored as a reducer slice — same `createFeatureSelector` and `createSelector` pattern.

- Topic 52 — NgRx Effects: Effects listen to `routerNavigationAction` to dispatch HTTP load actions, replacing the `ActivatedRoute` + `switchMap` pattern.
- Topic 53 — NgRx Selectors In Depth: Cross-slice selectors combining router state and feature state are the core Router Store pattern.

The single topic it extends most directly: Topic 38 — Angular Router. Router Store bridges Angular Router and NgRx Store, making the URL a first-class NgRx state slice.

Section 3 — Code Example

Setup — registering Router Store

```
// app.config.ts
export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes),
    provideStore({
      router: routerReducer, // Register routerReducer under 'router' key
    }),
    provideRouterStore(), // Connect Angular Router events to NgRx
  ],
};
```

Built-in Router Selectors

```
import { getRouterSelectors } from '@ngrx/router-store';

export const {
  selectUrl, // Current URL string: '/users/42'
  selectQueryParams, // { search: 'alice', page: '2' }
  selectRouteParams, // { id: '42' }
  selectRouteData, // Static data defined in route config
} = getRouterSelectors();
```

Effect that reacts to navigation

```
loadUserOnNavigation$ = createEffect(() =>
  this.actions$.pipe(
    ofType(routerNavigatedAction), // Navigation complete – component active
    concatLatestFrom(() => this.store.select(selectUserId)),
    filter(([ , userId]) => userId !== undefined),
    switchMap(([ , userId]) =>
      this.userService.getById(userId!).pipe(
        map(user => UsersApiActions.userLoaded({ user })),
        catchError(error =>
          of(UsersApiActions.usersLoadFailed({ error: error.message })))
```

```
)  
)  
)  
)  
);
```

Custom Router State Serializer

```
export class CustomRouterStateSerializer  
implements RouterStateSerializer<CustomRouterState> {  
  serialize(routerState: RouterStateSnapshot): CustomRouterState {  
    let route = routerState.root;  
    while (route.firstChild) { route = route.firstChild; }  
    return {  
      url: routerState.url,  
      params: route.params,  
      queryParams: routerState.root.queryParams,  
      data: route.data as Record<string, unknown>,  
    };  
  }  
}  
  
// Register in app.config.ts:  
provideRouterStore({ serializer: CustomRouterStateSerializer })
```

Section 4 — Before & After Refactor

Before — Direct `ActivatedRoute` injection in the component

Component subscribes to `route.paramMap`, owns HTTP logic, state is local to the component, other components cannot access 'the currently viewed user' without re-fetching, DevTools shows no navigation chain, no cancellation semantics.

After — Router Store + Effect pattern

Component is a pure view — reads signals, renders. Effect handles loading triggered by `routerNavigatedAction`. The loaded entity is in the global store — any component can select it without a second HTTP call. DevTools shows the full chain. `switchMap` handles cancellation automatically.

Section 5 — Common Mistakes & Misconceptions

Mistake 1 — Forgetting to register `routerReducer`

`provideRouterStore()` alone only connects Angular Router events to NgRx. It does not store router state. Must also register `routerReducer` in `provideStore({ router: routerReducer })`. Without

it, all router selectors return undefined silently.

Mistake 2 — Using `routerNavigationAction` when you need `routerNavigatedAction`

`routerNavigationAction` fires when guards have passed but the component hasn't activated yet. `routerNavigatedAction` fires when navigation is fully complete. For data loading, always use `routerNavigatedAction` — the component is guaranteed to be active and route params reflect the new route.

Mistake 3 — Not using a custom serializer

The default serializer stores the full `RouterStateSnapshot`, which contains component class references and injector instances that cannot be JSON-serialized. With `strictStateSerializability: true`, the app throws on every navigation. Always register a `CustomRouterStateSerializer`.

Section 6 — Do's and Don'ts

DO	DON'T
Register <code>routerReducer</code> in <code>provideStore({ router: routerReducer })</code>	Call <code>provideRouterStore()</code> without <code>routerReducer</code> — selectors return undefined
Always write a custom <code>RouterStateSerializer</code>	Use the default serializer — stores un-serializable Angular objects
Use <code>routerNavigatedAction</code> for data loading effects	Use <code>routerNavigationAction</code> — component isn't mounted yet
Use <code>concatLatestFrom</code> to read router state in effects	Use <code>withLatestFrom</code> — timing race with action-reduction
Compose typed selectors on top of <code>getRouterSelectors()</code>	Read <code>selectRouteParams</code> directly in components and cast inline
React to route changes in Effects	Inject <code>ActivatedRoute</code> in components that also use <code>NgRx</code>

Section 7 — Debugging Tips

Bug 1 — All router selectors return undefined

Cause: `routerReducer` not registered in `provideStore()`, or key passed to `getRouterSelectors()` doesn't match the key in `provideStore()`. Fix: `provideStore({ router: routerReducer })` + `getRouterSelectors()` with default 'router' key.

Bug 2 — Effect fires on every navigation, not just the target route

Cause: `ofType(routerNavigatedAction)` matches every navigation. Fix: Add `filter(() => userId !== undefined)` or filter the URL with `selectUrl` to ensure effect only runs on the intended routes.

Bug 3 — `routerNavigatedAction` fires but `selectRouteParams` returns previous route's params

Cause: `withLatestFrom` reads the store at the moment of the outer emission — the router reducer may not have processed the action yet. Fix: Use `concatLatestFrom` from `@ngrx/effects` instead — it reads state after the action has been fully reduced.

Section 8 — Real-World Applications

Scenario 1 — Master-detail navigation with shared entity state

A CRM contacts list and detail view share the same NgRx entity state. `loadContactOnNavigation$` Effect listens to `routerNavigatedAction`, reads `selectContactId`, checks if entity is already cached (avoids re-fetching), and dispatches `contactLoaded`. The list component can select the same entity without a second HTTP call.

Scenario 2 — URL-driven filter and pagination state

A products page at `/products?category=shoes&page=3&sort=price` keeps all filter state in the URL. `selectProductFilters` derives typed `{ category, page, sort }` from `selectQueryParams`. An Effect reloads products on `routerNavigatedAction` when on the `/products` route. The component updates filters by calling `router.navigate` with `queryParamsHandling: 'merge'` — never dispatching to the store.

Section 9 — Practice Exercises

Beginner

Set up NgRx Router Store in an existing Angular app. Register `routerReducer`, call `provideRouterStore()`, and write a custom `RouterStateSerializer` extracting url, params, and queryParams. Create three selectors: `selectCurrentUrl`, `selectRouteId` (string | undefined), and `selectSearchTerm` (string, defaulting to "").

Intermediate

Build a `PostDetailComponent` for `/posts/:id` without injecting `ActivatedRoute` in the component. Write an Effect listening to `routerNavigatedAction` using `concatLatestFrom` to read `selectPostId`, filtering out navigations without a post ID, dispatching `postLoaded` or `postLoadFailed`. Component reads `selectCurrentPost` (cross-slice selector) with `OnPush`.

Advanced

Build URL-driven search and pagination for `/users` route with `?search=&page=&pageSize=` query params. All filter state lives in the URL — store never stores search, page, or pageSize directly. Effect listens to `routerNavigatedAction`. Component updates filters via `router.navigate`. Write two unit tests: one verifying the effect fires with correct params, one verifying the selector extracts and types params correctly.

Section 10 — Key Takeaways

- Router Store connects Angular Router navigation events to NgRx — every navigation dispatches an action and stores serialized route state
- `routerReducer` stores router state; `getRouterSelectors()` reads it as memoised selectors

- `routerNavigatedAction` is the correct trigger for data loading — navigation and component activation both complete
- A custom `RouterStateSerializer` is required in any production app to prevent DevTools serialization errors
- `concatLatestFrom` (not `withLatestFrom`) is the safe way to read router state inside Effects reacting to router actions

One-Line Memory Aid: Router Store is the GPS dispatcher for NgRx — every navigation is logged as an action, route state is selectable like any store slice, and Effects replace direct `ActivatedRoute` injection by reacting to `routerNavigatedAction`.

Section 11 — Thought-Provoking Question + Answer

Question: If the URL already contains all filter and param information, and `ActivatedRoute` makes it accessible inside any component, what does Router Store actually add?

Answer: `ActivatedRoute` is sufficient for self-contained components with no cross-feature state sharing. Router Store becomes genuinely valuable in four situations: (1) Cross-slice selectors — route params need to combine with entity state in `createSelector`; `ActivatedRoute` is not accessible inside `createSelector`. (2) Effects as data-loading orchestrators — `routerNavigatedAction` is the trigger; without Router Store, Effects have no clean way to know the active route. (3) DevTools and time-travel debugging — navigation is visible in the action log with full context. (4) Testing — testing a component with `MockStore` needs only one line to preset `selectCurrentUser`; no `RouterTestingModule` required.

Practical rule: In fully NgRx apps where Effects own all data loading, Router Store is not redundant — it's the missing piece that makes Effects the single data-loading entry point.

Section 12 — Related Topics to Learn Next

- ES6: JSON serialization constraints — why custom `RouterStateSerializer` is required
- TypeScript: Discriminated unions — the five router action types form a discriminated union
- Topic 58 — NgRx Testing: Testing Router Store effects requires dispatching router actions to `MockStore`
- Topic 59 — NgRx Best Practices: Router Store fits into the URL-as-single-source-of-truth architectural pattern
- Topic 39 — Route Guards (review): `routerCancelAction` fires when a guard cancels navigation

Section 13 — Study Plan Checkpoint

Topics this depends on: Topic 38 (Angular Router), Topic 39 (Route Guards), Topic 51 (NgRx Core), Topic 52 (Effects), Topic 53 (Selectors), Topic 54 (Entity).

Topics this unlocks: Topic 58 (NgRx Testing — testing Router Store effects), Topic 59 (NgRx Best Practices).

Production readiness: NgRx Router Store is production-ready — requires custom serializer and `concatLatestFrom` discipline to be production-safe.

Section 14 — Common Interview Questions

Q1 (Mid): What is NgRx Router Store, what problem does it solve, and how do you set it up?

Strong answer: Problem — `ActivatedRoute` in components creates dual source of truth; Effects have no clean way to react to navigation; route state is invisible to DevTools. Solution — dispatches navigation as NgRx actions, stores route state as selectable NgRx slice. Setup: `provideStore({ router: routerReducer })` + `provideRouterStore()` + `CustomRouterStateSerializer`. Primary pattern: Effects listen to `routerNavigatedAction` + `concatLatestFrom(selectRouteParam)`.

Weak answers miss: The custom serializer requirement.

Q2 (Mid): Difference between `routerNavigationAction` and `routerNavigatedAction`?

Strong answer: `routerNavigationAction` fires when guards passed but component hasn't activated. `routerNavigatedAction` fires when navigation is fully complete — component is mounted and params are finalized. For data loading always use `routerNavigatedAction`. `routerNavigationAction` risk: loads data for routes that guards may still cancel.

Q3 (Senior): Why use `concatLatestFrom` instead of `withLatestFrom` when reading router state inside an Effect reacting to a router action?

Strong answer: `withLatestFrom` reads the store at the moment the outer Observable emits — which is the `routerNavigatedAction` dispatch. The router reducer processes the same action to update the store — timing window where action dispatched but state not yet reduced. `concatLatestFrom` defers state read until after action is fully reduced. Symptom: navigating from `/users/42` to `/users/99`, effect reads `id: '42'` because reducer hasn't run yet.

Section 15 — Anti-Patterns to Avoid in Production

Anti-Pattern 1 — Using the default Router State Serializer in production

The `DefaultRouterStateSerializer` stores the full Angular `RouterStateSnapshot` containing component class references and circular references. With `strictStateSerializability: true`, app throws on every navigation. Redux DevTools silently fails for time-travel. Always register `CustomRouterStateSerializer`.

Anti-Pattern 2 — Mixing `ActivatedRoute` and Router Store in the same component

Two systems now own the same information. If Router Store serializer timing differs from `ActivatedRoute.params` on complex navigations, the component sees inconsistent state. Testing requires both `RouterTestingModule` and `MockStore`. As codebase grows, no clear authority on which source is correct.

Anti-Pattern 3 — Dispatching a load action from a component on navigation instead of using an Effect

Component subscribes to `ActivatedRoute` and dispatches. Component owns route-observation logic — harder to test, harder to move to different route structure, impossible to pre-fetch. If two components on same page both need navigation dispatch, both must independently subscribe. Effect-based pattern centralises all navigation reactions in one place.

Section 16 — Mental Model Summary

Core concept: `NgRx Router Store` logs every Angular navigation as an `NgRx` action and stores the current route state (`url`, `params`, `queryParams`) as a selectable `NgRx` slice — making the URL a first-class member of application state that `Effects` can react to and components can read without touching `ActivatedRoute`.

Rule	Detail
1. Both pieces required	<code>provideStore({ router: routerReducer })</code> AND <code>provideRouterStore()</code>
2. Custom serializer always	Default stores un-serializable Angular objects — breaks <code>DevTools</code>
3. <code>routerNavigatedAction</code>	For data loading — navigation and component activation both complete
4. <code>concatLatestFrom</code>	Not <code>withLatestFrom</code> — avoids action-reduction timing race
5. No <code>ActivatedRoute</code> in <code>NgRx</code>	<code>Router Store</code> selectors are the single source of truth for route state
6. Typed selectors	Compose and type-cast <code>params</code> once in selectors, reuse everywhere

Most important rule: Always write a custom `RouterStateSerializer` — the default stores Angular component references that cannot be JSON-serialized, breaking `Redux DevTools` and any `strictStateSerializability` runtime check.

TOPIC 58 — NgRx Testing (Store, Effects, Selectors, Component Store)

Angular & NgRx | Difficulty: Advanced | Angular Relevance: Critical

Section 0 — Difficulty & Priority Rating

Difficulty: Advanced — NgRx Testing requires holding five distinct testing contexts simultaneously (Store, Effects, Selectors, Component Store, SignalStore), each with its own setup pattern, mock strategy, and assertion style.

Angular Relevance: Critical — Every NgRx pattern learned in Topics 51-57 is only production-safe if tested; untested reducers silently mutate state, untested effects silently swallow errors, and untested selectors silently recompute on every action.

Section 1 — Explanation

The Flight Simulator Analogy

A commercial airline doesn't let pilots fly a new aircraft type by reading the manual and then boarding 300 passengers. They spend hours in a flight simulator — a controlled environment that mimics every system, lets the pilot make mistakes, and verifies the correct response to every scenario.

NgRx Testing is your flight simulator for application state. MockStore is the simulator cockpit — looks and feels like the real store but you control every instrument reading directly. provideMockActions is the simulated air traffic control — you feed exactly the actions you want the effect to receive. Selector projector testing is instrument calibration — you test the pure function directly. ComponentStore testing is a ground-level systems check.

Layer	Primary Tool	Key Setup
Reducer	Plain function call	reducer(initialState, action)
Selector	selector.projector(mockInputs)	No store, no TestBed
Store (component)	MockStore + provideMockStore()	TestBed.configureTestingModule
Effects	provideMockActions() + ReplaySubject	TestBed + mock services
Component Store	Real ComponentStore + mock service	TestBed with providers
SignalStore	Real SignalStore + mock service	TestBed with providers
Router Store effects	MockStore + dispatch routerNavigatedAction	overrideSelector for route params

Section 2 — Connection to Prior Concepts

- Topic 47 — Testing Angular Components: `fixture.detectChanges()`, `MockStore` in `TestBed`, `fakeAsync/tick`, `data-testid` querying — all patterns now applied to `NgRx`-connected components.
- Topic 48 — Testing Services & HTTP: `HttpClientTestingModule`, `httpController.verify()`, `req.flush()` — directly reused when testing `NgRx` Effects that make HTTP calls.
- Topic 51 — `NgRx` Core Concepts: Reducers are pure functions — testing them requires no Angular setup, just `reducer(state, action)`.
- Topic 52 — `NgRx` Effects: Effects are the most complex layer to test — `ReplaySubject`, `provideMockActions`, and inner/outer `Observable` structure are the testing surface.
- Topic 53 — `NgRx` Selectors In Depth: `selector.projector()` tests the pure derivation function directly.

The single topic it extends most directly: Topic 48 — Testing Services & HTTP. `NgRx` Effects are services that make HTTP calls — the `HttpClientTestingModule` + `req.flush()` pattern from Topic 48 is applied directly inside Effect tests.

Section 3 — Code Example

Testing 1 — Reducer (no Angular setup needed)

```
describe('usersReducer', () => {  
  const initialState = { users: [], loading: false, error: null };  
  
  it('should return initial state for unknown action', () => {  
    const result = usersReducer(initialState, { type: 'UNKNOWN' } as any);  
    expect(result).toBe(initialState); // toBe = same reference  
  });  
  
  it('should store users on usersLoaded', () => {  
    const users = [{ id: '1', name: 'Alice', email: '' }];  
    const result = usersReducer(  
      { ...initialState, loading: true },  
      UsersApiActions.usersLoaded({ users })  
    );  
    expect(result.users).toEqual(users);  
    expect(result.loading).toBe(false);  
    expect(result).not.toBe(initialState); // Must return NEW object  
  });  
});
```

Testing 2 — Selectors via `selector.projector()`

```
describe('User Selectors', () => {
```

```

const users = [
  { id: '1', name: 'Alice', email: 'alice@admin.com' },
  { id: '2', name: 'Bob', email: 'bob@example.com' },
];

it('selectAllUsers projector should return users from state slice', () => {
  // No store, no TestBed – just a function call
  const result = selectAllUsers.projector({ users, loading: false, error: null });
  expect(result).toEqual(users);
});

it('selectAdminUsers projector should filter to admin emails', () => {
  // Input is User[] (output of selectAllUsers), not UsersState
  const result = selectAdminUsers.projector(users);
  expect(result).toHaveLength(1);
  expect(result[0].name).toBe('Alice');
});

```

Testing 3 — NgRx Store in component test with MockStore

```

describe('UsersComponent', () => {
  let store: MockStore;

  beforeEach(async () => {
    await TestBed.configureTestingModule({
      imports: [UsersComponent],
      providers: [provideMockStore({ initialState: {} })],
    }).compileComponents();
    store = TestBed.inject(MockStore);
  });

  afterEach(() => {
    store.resetSelectors(); // ALWAYS – prevents leaking between tests
  });

  it('should render users from store', () => {
    store.overrideSelector(selectAllUsers, mockUsers);
    store.overrideSelector(selectLoading, false);
    store.refreshState(); // Apply overrides – triggers emissions
    fixture.detectChanges(); // Update DOM
    const items = fixture.nativeElement
      .querySelectorAll('[data-testid="user-name"]');
  });

```

```
expect(items).toHaveLength(1);
});
});
```

Testing 4 — NgRx Effects

```
describe('UserEffects', () => {
  let effects: UserEffects;
  let actions$: ReplaySubject<Action>; // ReplaySubject not Subject!
  let userService: jasmine.SpyObj<UserService>;

  beforeEach(() => {
    actions$ = new ReplaySubject<Action>(1);
    const spy = jasmine.createSpyObj('UserService', ['getAll']);

    TestBed.configureTestingModule({
      providers: [
        UserEffects,
        provideMockActions(() => actions$),
        provideMockStore(),
        { provide: UserService, useValue: spy },
      ],
    });

    effects = TestBed.inject(UserEffects);
    userService = TestBed.inject(UserService) as jasmine.SpyObj<UserService>;
  });

  it('should dispatch usersLoaded on success', async () => {
    userService.getAll.and.returnValue(of(mockUsers));
    actions$.next(UsersPageActions.pageOpened());

    // Use firstValueFrom – never assert inside subscribe() without done
    const action = await firstValueFrom(effects.loadUsers$);
    expect(action).toEqual(UsersApiActions.usersLoaded({ users: mockUsers }));
  });

  it('should dispatch usersLoadFailed on HTTP error', async () => {
    userService.getAll.and.returnValue(
      throwError(() => new Error('Network error'))
    );
    actions$.next(UsersPageActions.pageOpened());
    const action = await firstValueFrom(effects.loadUsers$);
```

```

expect(action).toEqual(
UsersApiActions.usersLoadFailed({ error: 'Network error' })
);
});
});

```

Testing 5 — Component Store

```

describe('DataTableStore', () => {
let store: DataTableStore;
let userService: jasmine.SpyObj<UserService>;

beforeEach(() => {
const spy = jasmine.createSpyObj('UserService', ['getAll']);
TestBed.configureTestingModule({
providers: [DataTableStore, { provide: UserService, useValue: spy }],
});
store = TestBed.inject(DataTableStore);
userService = TestBed.inject(UserService) as jasmine.SpyObj<UserService>;
});

it('should initialize with empty state', () => {
expect(store.users()).toEqual([]); // Signal read with ()
expect(store.loading()).toBe(false);
});

it('should update users after loadUsers effect succeeds', (done) => {
userService.getAll.and.returnValue(of(mockUsers));
store.loadUsers();
setTimeout(() => { // Effects are async
expect(store.users()).toEqual(mockUsers);
done();
});
});
});

```

Section 4 — Before & After Refactor

Before — Testing a component with real Store and real reducers

Using `StoreModule.forRoot({ users: usersReducer })` in component tests: test is testing the reducer AND the component simultaneously — if reducer has a bug, component tests fail for unrelated reasons. Cannot test impossible state combinations. Test is slow.

After — MockStore with overrideSelector

provideMockStore({ initialState: {} }) with overrideSelector — set exactly the state the test needs, no reducer required. Any state combination injectable directly. Faster — no reducer computation. resetSelectors() in afterEach prevents leaking.

Section 5 — Common Mistakes & Misconceptions

Mistake 1 — Asserting inside subscribe without done() — the silent green test

Jasmine marks the test complete before the async callback runs. The assertion never executes. The test passes (green) even if the Effect produces nothing. Fix: Always use done callback or firstValueFrom() with async/await. This is Topic 48's most critical rule applied to Effects.

Mistake 2 — Forgetting store.resetSelectors() in afterEach

overrideSelector is persistent — not per-test. Selector overrides from test A are active in test B. Tests become order-dependent — pass in isolation but fail in full suite runs. Always call store.resetSelectors() in afterEach.

Mistake 3 — Testing selector composition logic through MockStore instead of projector()

Using store.setState() and store.select() to test selectors conflates selector registration with derivation logic. Test is slower, requires TestBed for a pure function test, and hides the actual input-output contract. Always use selector.projector(mockInputs) — no store, no TestBed.

Section 6 — Do's and Don'ts

DO	DON'T
Test reducers as reducer(state, action) — no Angular	Import StoreModule.forRoot() in reducer tests
Test selectors via selector.projector(mockInputs)	Test selectors by setting up MockStore state
Use MockStore + overrideSelector in component tests	Use real StoreModule.forRoot() in component tests
Always call store.resetSelectors() in afterEach	Omit resetSelectors() — overrides leak between tests
Use done or firstValueFrom() for Effect assertions	Assert inside subscribe() without done — silent false green
Use ReplaySubject<Action>(1) for mock actions	Use Subject<Action> — if Effect subscribes after action emits, it misses it

Section 7 — Debugging Tips

Bug 1 — Effect test is green but the assertion never runs

Symptom: Removing the expect() line also passes. Cause: Assertion inside subscribe() without done parameter. Jasmine exits before Observable emits. Fix: Use async/await with firstValueFrom(), or add done to test function.

Bug 2 — MockStore selector returns undefined despite overrideSelector

Cause: `store.refreshState()` was not called after `overrideSelector()`. Fix: Always call `store.overrideSelector()` then `store.refreshState()` then `fixture.detectChanges()` in sequence.

Bug 3 — Component Store test shows stale state after calling an effect method

Cause: `rxMethod` and Component Store's `effect()` are asynchronous — the HTTP Observable resolves on the next microtask. Fix: Wrap assertion in `setTimeout(() => { expect(...); done(); })` or use `fakeAsync + tick()` if the service returns a synchronous Observable.

Section 8 — Real-World Applications

Scenario 1 — Testing an optimistic update Effect

An optimistic delete Effect removes a contact immediately, calls the API, and restores if it fails. Test collects all dispatched actions by subscribing to `effects.deleteContact$`, then verifies: first action is `contactRemovedOptimistically`, second action is `contactRestored`. Uses `done` callback and subscribe with complete handler.

Scenario 2 — Testing a SignalStore with `withHooks.onInit`

`withHooks.onInit` fires automatically when the store is provided in `TestBed`. Configure the mock service return value BEFORE `TestBed.inject(UserStore)` — `onInit` fires during injection. To prevent auto-load in tests needing clean state: configure `spy` to return empty before injection, reset after.

Section 9 — Practice Exercises

Beginner

Write a complete test suite for `usersReducer` covering: (1) unknown action returns same state reference (`toBe` not `toEqual`); (2) `usersLoaded` stores users and sets `loading: false`; (3) `usersLoadFailed` stores the error message. Then write projector tests for `selectAllUsers` and `selectAdminUsers` — no `TestBed`, no store, just function calls.

Intermediate

Write a `MockStore`-based test suite for `UsersComponent` that: (1) dispatches `UsersPageActions.pageOpened()` in `ngOnInit`; (2) renders `[data-testid='user-name']` for each user; (3) shows `[data-testid='loading']` when `selectLoading` is true; (4) hides it when false. Use `overrideSelector` and `refreshState`. Include `resetSelectors()` in `afterEach`.

Advanced

Write a full Effect test suite for a `deleteUser$` Effect using `concatMap` that: (1) dispatches `userDeletedOptimistically` immediately; (2) calls `userService.delete(id)`; (3) dispatches `userDeleteConfirmed` on success; (4) dispatches `userRestored` on failure; (5) verifies that two quick delete actions are queued by `concatMap` rather than cancelled. Use `ReplaySubject`, `provideMockActions`, `firstValueFrom` for all assertions.

Section 10 — Key Takeaways

- Reducers are pure functions — test with `reducer(state, action)`, no Angular required
- Selector projectors are pure functions — test with `selector.projector(mockInputs)`, no store required
- Component tests use `MockStore + overrideSelector` to control state precisely
- Effect tests use `provideMockActions(() => actions$)` with `ReplaySubject<Action>`
- Component Store and `SignalStore` are tested with real store class + mocked service in `TestBed`
- Always call `store.resetSelectors()` in `afterEach` — selector overrides persist between tests
- Always use `done`, `firstValueFrom()`, or `fakeAsync+tick()` for async assertions — never assert inside `subscribe()` without `done`

Layer	Tool	Async handling
Reducer	<code>reducer(state, action)</code>	Synchronous
Selector	<code>selector.projector(inputs)</code>	Synchronous
Component	<code>MockStore + overrideSelector</code>	<code>detectChanges()</code> after <code>refreshState()</code>
Effect	<code>ReplaySubject + provideMockActions</code>	<code>done</code> / <code>firstValueFrom</code>
Component Store	Real store + mock service	<code>fakeAsync+tick</code> or <code>done+setTimeout</code>
<code>SignalStore</code>	Real store + mock service	<code>fakeAsync+tick</code> or <code>done+setTimeout</code>

One-Line Memory Aid: Test reducers and selector projectors as plain functions; test Effects with `ReplaySubject + provideMockActions`; test Component Store and `SignalStore` with the real store class and a mocked service — and always use `done` or `firstValueFrom` for async assertions or your tests are silently lying.

Section 11 — Thought-Provoking Question + Answer

Question: If `MockStore.overrideSelector()` bypasses the real selector logic and returns a hardcoded value, does that mean a completely broken selector makes all component tests still pass?

Answer: Yes — and this is a deliberate design trade-off. `MockStore` tests the component in isolation: how the component renders given a particular state. The selector's correctness is a separate concern tested via `selector.projector()`.

The separation mirrors mocking a service in component tests (Topic 47) — you mock `UserService.getAll()` so the component test isn't sensitive to service implementation details. `overrideSelector` does the same for selectors.

Practical rule: For every `createSelector()` call, there should be at least one `selector.projector()` test covering the derivation logic. For every component that subscribes to a selector, use `overrideSelector` in the component test. The two test types are complementary, not redundant.

Section 12 — Related Topics to Learn Next

- RxJS: ReplaySubject vs Subject vs BehaviorSubject — why ReplaySubject(1) is required for mock actions
- RxJS: firstValueFrom() vs toPromise() — firstValueFrom is the modern RxJS 7+ way for async/await test assertions
- TypeScript: jasmine.SpyObj<T> typing — strongly-typed spies catch mismatched method names at compile time
- Topic 59 — NgRx Best Practices: Testing strategy (what to test, what to mock, where to draw the line)
- Topic 47 (review): detectChanges, fakeAsync, data-testid — the DOM-side of every NgRx component test
- Topic 48 (review): HttpClientTestingModule + req.flush() — the HTTP-side of every Effect test

Section 13 — Study Plan Checkpoint

Topics this depends on: Topic 47 (Testing Components), Topic 48 (Testing Services & HTTP), Topic 51 (NgRx Core), Topic 52 (Effects), Topic 53 (Selectors), Topic 55 (Component Store), Topic 56 (SignalStore), Topic 57 (Router Store).

Topics this unlocks: Topic 59 — NgRx Best Practices (testing strategy is a core part of NgRx production architecture).

Production readiness: The patterns in this topic are the minimum viable NgRx test suite for any production Angular app — teams without these tests are shipping untested state management, the highest-risk category of silent bug.

Section 14 — Common Interview Questions

Q1 (Mid): How do you test an NgRx Effect? Walk me through the setup and explain why each piece is needed.

Strong answer: `ReplaySubject<Action>(1)` for `actions$` — replays last emission to late-subscribing Effects; Subject loses emissions if Effect subscribes after dispatch. `provideMockActions(() => actions$)` — connects `ReplaySubject` to the Effect's Actions injection token. `provideMockStore()` — required if Effect uses `withLatestFrom` to read store state. Mock service via `{ provide: RealService, useValue: spyObj }`. `done` callback or `firstValueFrom()` for assertions. Test both happy and error paths.

Weak answers miss: The `ReplaySubject` vs `Subject` distinction.

Q2 (Mid): What is `selector.projector()` and why use it instead of testing selectors through `MockStore`?

Strong answer: `projector()` calls the selector's pure derivation function directly with mock inputs — no store, no `TestBed`. Projector inputs are the outputs of the selector's input selectors — for `selectAdminUsers(selectAllUsers)`, input is `User[]`, not `UsersState`. Testing via `MockStore` conflates registration with derivation logic, is slower, and harder to debug.

Weak answers miss: What the projector inputs actually are.

Q3 (Senior): Your NgRx Effect test suite shows 100% pass rate but an Effect silently does nothing after the first HTTP error. What went wrong?

Strong answer: Root cause — `catchError` on the outer Observable permanently kills the Effect. Test failure — only tested the error path once, never tested 'fire twice with second call succeeding after first errored.' Missing test: after the error-path test, emit a second success action and assert the Effect dispatches the success action — proving the outer stream survived. Code fix: move `catchError` inside the `switchMap` inner Observable.

Weak answers miss: The 'stays alive after error' test pattern.

Section 15 — Anti-Patterns to Avoid in Production

Anti-Pattern 1 — Using `StoreModule.forRoot()` in component tests

Using real store in component tests: test is an integration test. When it fails, unknown which layer broke. Setting up complex initial state requires dispatching multiple actions in sequence. Cannot test edge-case rendering with impossible state. Test suite is slow and brittle. Always use `MockStore`.

Anti-Pattern 2 — Omitting `store.resetSelectors()` in `afterEach`

Selector overrides accumulate across tests. Tests become order-dependent — pass in isolation but fail in full suite runs. In CI where test order may differ from local, bugs hide for weeks. This is the most common source of flaky tests in NgRx projects.

Anti-Pattern 3 — Testing the full NgRx stack in a single integration test instead of each layer independently

One end-to-end test covering action → effect → reducer → selector → component: when it fails, unknown which layer broke. No diagnostic value. Slow. Fragile. Fix: `reducer.spec.ts` (pure functions), `selectors.spec.ts` (`projector()`), `effects.spec.ts` (`provideMockActions` + mock service), `component.spec.ts` (`MockStore` + `overrideSelector`), e2e/ for full integration.

Section 16 — Mental Model Summary

Core concept: NgRx Testing has five distinct layers — each tested independently with its own tool: reducers and selector projectors as plain functions, Effects with `ReplaySubject` + `provideMockActions`, components with `MockStore` + `overrideSelector`, and Component Store / `SignalStore` with the real store class and mocked services.

Rule	Detail
1. Reducers as functions	reducer(state, action) — toBe for unchanged ref, toEqual for changed
2. Selector projectors	selector.projector(mockInputs) — input is output of upstream selectors
3. MockStore for components	Never StoreModule.forRoot() in unit tests
4. resetSelectors() always	afterEach — overrides are persistent across tests
5. ReplaySubject for actions	Not Subject — Effects may subscribe after action emits
6. done or firstValueFrom	Never assert inside subscribe() without done — silent false green

Most important rule: Never assert inside subscribe() without a done callback — Jasmine exits before the async callback executes, the assertion never runs, and the test is permanently green regardless of whether the Effect works correctly.

TOPIC 59 — NgRx Best Practices & Architecture Patterns

Angular & NgRx | Difficulty: Advanced | Angular Relevance: Critical

Section 0 — Difficulty & Priority Rating

Difficulty: Advanced — This topic requires holding the entire NgRx ecosystem in your head simultaneously and making principled architectural decisions under real-world constraints; the difficulty is not any single new API but the judgment required to apply six topics of NgRx knowledge correctly at scale.

Angular Relevance: Critical — Every NgRx pattern learned in Topics 51-58 can be applied well or badly; this topic is the decision framework that determines which — teams that skip architectural discipline end up with NgRx stores harder to maintain than the component state they replaced.

Section 1 — Explanation

The City Planning Analogy

A city needs infrastructure — roads, utilities, zoning laws. A small village can be built informally: houses wherever there's space, roads connecting them ad hoc. That works at small scale. But as the city grows, informal decisions compound into problems.

NgRx architecture is city planning for application state. At small scale, any approach works. At team scale and production scale, the decisions you make about where state lives, how it flows, what gets named what, and where side effects are allowed determine whether your codebase is a well-planned city or an informal sprawl.

The NgRx Decision Matrix

State characteristic	Recommended tool
Shared across multiple features	Global NgRx Store
Survives navigation (persistent)	Global NgRx Store
Needs DevTools / time-travel	Global NgRx Store
Collection of identifiable records	NgRx Entity (Topic 54)
URL-driven state	NgRx Router Store (Topic 57)
Complex local UI state (3+ slices, async)	NgRx SignalStore or Component Store
Simple local UI state (1-2 values)	Angular Signals or signal()
One-off flag or toggle	Local signal() in component

Section 2 — Connection to Prior Concepts

This topic directly builds on every NgRx topic in the series:

- Topic 51 — NgRx Core Concepts: Action naming discipline, reducer purity rules, one-way data flow
- Topic 52 — NgRx Effects: Operator selection rules (switchMap for reads, concatMap for writes, exhaustMap for submissions) and catchError placement inside inner Observables
- Topic 53 — NgRx Selectors: Layered selector pattern, one view model selector per component, never returning new literals from projectors
- Topic 54 — NgRx Entity: Entity is the correct structure for any collection of records
- Topics 55 & 56 — Component Store & SignalStore: The boundary between local and global state
- Topic 57 — Router Store: URL-as-state-of-truth principle and Effect-driven data loading
- Topic 58 — NgRx Testing: Testability as an architectural constraint

The single topic it extends most directly: Topic 51 — NgRx Core Concepts. Every best practice is an extension of Topic 51's foundational discipline.

Section 3 — Code Example — Complete Production Feature Architecture

Feature folder structure (the standard layout)

```
src/app/features/orders/  
■■■ orders.routes.ts ← Lazy route with provideState + provideEffects  
■■■ orders.actions.ts ← createActionGroup per source  
■■■ orders.reducer.ts ← createFeature + EntityAdapter  
■■■ orders.selectors.ts ← Layered selectors + one vm selector per component  
■■■ orders.effects.ts ← All side effects; no HTTP in components  
■■■ orders.service.ts ← HTTP only; no store access  
■■■ components/  
■■■ orders-list/  
■■■ order-detail/
```

Actions — one createActionGroup per source, past-tense naming

```
// CORRECT: past-tense event names  
export const OrdersPageActions = createActionGroup({  
  source: 'Orders Page',  
  events: {  
    'Page Opened': emptyProps(),  
    'Order Selected': props<{ orderId: string }>(),  
    'Create Order Clicked': props<{ order: Omit<Order, 'id'> }>(),  
    'Delete Order Clicked': props<{ orderId: string }>(),  
  },  
});
```

```

export const OrdersApiActions = createActionGroup({
  source: 'Orders API',
  events: {
    'Orders Loaded': props<{ orders: Order[] }>(),
    'Order Created': props<{ order: Order }>(),
    'Order Deleted': props<{ orderId: string }>(),
  },
});

// NEVER: 'Load Orders', 'Delete Order', 'Set Loading'

```

Effects — correct operator selection

```

// READ – switchMap: cancel previous load
loadOrders$ = createEffect(() =>
  this.actions$.pipe(
    ofType(OrdersPageActions.pageOpened),
    switchMap(() => // switchMap for reads only
      this.ordersService.getAll().pipe(
        map(orders => OrdersApiActions.ordersLoaded({ orders })),
        catchError(err => // catchError INSIDE switchMap
          of(OrdersApiActions.ordersLoadFailed({ error: err.message })))
        )
      )
    )
);

// WRITE – concatMap: queue creates, never cancel
createOrder$ = createEffect(() =>
  this.actions$.pipe(
    ofType(OrdersPageActions.createOrderClicked),
    concatMap(({ order }) => // concatMap for ordered writes
      this.ordersService.create(order).pipe(
        map(created => OrdersApiActions.orderCreated({ order: created })),
        catchError(err =>
          of(OrdersApiActions.orderCreateFailed({ error: err.message })))
        )
      )
    )
);

```


Lazy route registration

```
export const ORDERS_ROUTES: Routes = [{
  path: '',
  canActivate: [authGuard],
  providers: [
    provideState(ordersFeature), // Lazy – loads only when route activates
    provideEffects(OrdersEffects),
  ],
  children: [
    {
      path: '',
      loadComponent: () => import('./orders-list.component')
        .then(m => m.OrdersListComponent),
    },
    {
      path: ':orderId',
      loadComponent: () => import('./order-detail.component')
        .then(m => m.OrderDetailComponent),
    },
  ],
}];
```

Component — thin view layer

```
@Component({
  changeDetection: ChangeDetectionStrategy.OnPush, // Always with NgRx
  template: `
    @let vm = orderVm();
    @if (vm.loading) { <app-spinner /> }
    @for (order of vm.orders; track order.id) {
      <app-order-card [order]='order' (delete)='onDelete(order.id)' />
    }
  `
})
export class OrdersListComponent {
  private store = inject(Store);
  orderVm = this.store.selectSignal(selectOrdersListVm); // ONE selector

  ngOnInit() { this.store.dispatch(OrdersPageActions.pageOpened()); }
  onDelete(id: string) {
    this.store.dispatch(OrdersPageActions.deleteOrderClicked({ orderId: id }));
  }
}

// Zero HTTP. Zero service injection. Pure view.
```

```
}
```

Section 4 — Before & After Refactor

Before — Component making HTTP calls directly alongside NgRx

Component injects both Store and OrdersService. HTTP called directly in ngOnInit — bypasses Effects entirely. DevTools shows no effect chain. Hand-rolled action objects bypass TypeScript safety. Error handling is component's responsibility — inconsistent across app. No cancellation semantics.

After — Correct architecture

Component dispatches past-tense events only. Effect owns: switchMap cancellation, HTTP call, error handling, success dispatch. DevTools shows: [Orders Page] Page Opened -> [Orders API] Orders Loaded — full audit trail. Component is testable with only MockStore.

Section 5 — Common Mistakes & Misconceptions

Mistake 1 — Putting everything in the global store 'just in case'

Every piece of UI state — modal open/closed, accordion expanded, tab index, form step — added to the global NgRx store. The store grows to hundreds of slices, mostly ephemeral UI state. DevTools becomes noise. Reducers and selectors multiply for zero architectural benefit. Rule: 'Does anything outside this component need this value?' If no — use local signal().

Mistake 2 — Writing command-style actions instead of event-style actions

Commands describe what to do (LOAD_ORDERS, SET_LOADING). Events describe what happened (Orders Loaded, Page Opened). Events decouple the dispatcher from the handler — the component doesn't know or care what happens as a result. One event can be handled by multiple reducers AND multiple effects simultaneously.

Mistake 3 — Using switchMap for write operations

switchMap for DELETE silently cancels in-flight delete requests on double-click. The server never receives the first delete, but the UI already removed the item optimistically. Data corruption in production. No error, no console warning. Always use concatMap for ordered writes.

Section 6 — Do's and Don'ts

DO	DON'T
Use createActionGroup with one group per source	Hand-roll action objects — no TypeScript safety, no DevTools grouping
Name actions as past-tense events (Orders Loaded)	Name actions as commands (Load Orders, Set Loading)
Put ALL side effects in Effects	Call services directly in components alongside dispatch()

DO	DON'T
Use concatMap for writes (DELETE, POST, PUT)	Use switchMap for write operations — silently cancels in-flight requests
Register feature state lazily in route providers	Register all feature state in app.config.ts
Apply OnPush to every NgRx-connected component	Leave components on Default change detection

Section 7 — Debugging Tips

Bug 1 — Store grows unboundedly and DevTools becomes unusable

Symptom: Hundreds of slices, thousands of [Modal] Opened and [Tab] Changed actions drowning meaningful changes. Cause: All UI state in global store; feature state registered eagerly. Fix: Audit each slice against the decision matrix. Move ephemeral UI state to local signal(). Register feature state lazily.

Bug 2 — Selector memoisation breaks and OnPush components re-render constantly

Symptom: OnPush components re-render on every store action, even unrelated ones. Cause: A selector projector returns new object or array literal ({} or []) on every call. Fix: Audit every createSelector projector for {} and [] returns. Sort in the reducer via EntityAdapter.sortComparer, not in selectors.

Bug 3 — An Effect permanently stops working after the first error

Symptom: Effect works for first user action, then silently stops responding. No console error. Cause: catchError on the outer Observable — one error permanently kills the Effect. Fix: Move catchError inside the switchMap/concatMap/exhaustMap callback — it must catch errors from the inner Observable only.

Section 8 — Real-World Applications

Scenario 1 — Scaling NgRx across a large team with a shared architectural standard

A team of 15 developers adopts the EARS pattern: Entity adapter for all collections, Action groups per source, Router Store for navigation-driven loads, SignalStore for local feature UI state. Enforced with @ngrx/eslint-plugin rules: no-dispatch-in-effects, prefer-action-creator, prefer-selector-in-select, prefer-concat-latest-from. ESLint enforces patterns mechanically — developers get lint errors before PRs are merged.

Scenario 2 — Incremental NgRx adoption in an existing Angular app

Phase 1: Identify shared state pain points — which state passes down 5+ levels of @Input()? Phase 2: Add NgRx for one painful feature (auth). Start with provideStore({}) + provideEffects([]). Phase 3: Add Entity for collections as features are touched. Phase 4: Add Router Store after 3+ features are in NgRx. Phase 5: Replace component-level state with SignalStore. NgRx adoption does not have to be all-or-nothing.

Section 9 — Practice Exercises

Beginner

You have actions: `on(SomeActions.loadData, state => ({ ...state, data: [] })), on(SomeActions.setLoading, (state, { loading }) => ({ ...state, loading })), on(SomeActions.setError, (state, { error }) => ({ ...state, error })))`. Identify three architectural problems and refactor using correct naming convention, loading flag management, and `createActionGroup`.

Intermediate

Audit this Effect and identify all violations (there are at least five): `loadAndSave$ = createEffect(() => actions$.pipe(ofType('[Orders] Load'), switchMap(() => ordersService.getAll()), tap(orders => store.dispatch({ type: '[Orders] Set Data', orders })), switchMap(orders => analyticsService.track('orders_loaded').pipe(catchError(() => EMPTY)))).pipe({ dispatch: false })`. Name each violation, explain why it's wrong, provide corrected code.

Advanced

Design the complete NgRx architecture for a real-time collaborative document editor: (1) load document on navigation to `/documents/:docId`; (2) real-time changes via WebSocket; (3) track local unsaved changes separately from server-confirmed state; (4) optimistic saves with restore on failure; (5) `DocumentEditorSignalStore` for local UI state separate from global document state. Produce: state interface, action groups, selector layers, effect list with correct operators, and what goes in global store vs `SignalStore`.

Section 10 — Key Takeaways

- NgRx is the right tool for shared, persistent, cross-feature state — not for all state in the application
- Actions are events (past-tense, `[Source] Something Happened`) not commands — the foundation of NgRx event-driven architecture
- Effects own all side effects — components dispatch events and read signals, nothing else
- Operator selection: `switchMap` for reads only, `concatMap` for ordered writes, `exhaustMap` for submissions
- `catchError` must always be inside the inner Observable — on the outer stream it permanently kills the Effect
- Feature state is always registered lazily in route providers — never eagerly in `app.config.ts`
- `ChangeDetectionStrategy.OnPush` is mandatory for every NgRx-connected component

The Seven Laws of NgRx Architecture

Law	Rule
1. State placement	Global for shared/persistent; local signal/ <code>SignalStore</code> for ephemeral
2. Action naming	Past-tense events with <code>[Source] Event Name</code> — never commands

Law	Rule
3. Effect ownership	All side effects in Effects — never in components
4. Operator selection	switchMap=reads, concatMap=ordered writes, exhaustMap=submissions
5. Error handling	catchError inside inner Observable — never on outer stream
6. State registration	Lazy via provideState() in route providers — never eager
7. Change detection	OnPush on every NgRx-connected component — mandatory

One-Line Memory Aid: NgRx architecture has seven laws — break any one and the store works but doesn't scale.

Section 11 — Thought-Provoking Question + Answer

Question: If NgRx adds so much boilerplate, is there a point at which the overhead simply isn't worth it?

Answer: Yes — any honest NgRx advocate will say so. NgRx is worth it when three or more of these are true: the app has multiple features that share state, the team has three or more developers working concurrently, state changes need to be auditable, the app has complex async sequences, the app will grow significantly over time.

NgRx is likely overkill when: single-developer project or small prototype, state is primarily local to individual pages, team is not already familiar with RxJS, app is content-heavy with minimal user interaction.

The modern middle ground: Angular 17+ with Signals and SignalStore covers a significant portion of what used to require NgRx. For apps that don't need global shared state, DevTools time-travel, or complex Effect orchestration, signal() + SignalStore + a clean service layer is a legitimate architecture.

Practical rule: Start without NgRx. Add it when you feel one of these specific pains: 'I need to share this state across three features,' or 'I can't debug production state,' or 'I need to cancel in-flight requests.' Add NgRx to solve pain you feel — not to preemptively architect for scale you haven't reached.

Section 12 — Related Topics to Learn Next

- ES6: Module systems and tree-shaking — verify lazy-loaded NgRx feature state actually reduces bundle size
- TypeScript: Discriminated unions — NgRx actions form a discriminated union
- Angular DevTools — Signal graph view complements Redux DevTools; together give full visibility
- Micro-frontends with NgRx — at very large scale, multiple Angular apps sharing NgRx state
- @ngrx/data — higher-level abstraction over NgRx Entity generating actions, reducers, effects, and selectors automatically

Section 13 — Study Plan Checkpoint

Topics this depends on: The entire NgRx series — Topics 51 through 58. Every NgRx concept is referenced in this topic.

Topics this unlocks: This is Topic 59 — the final topic in the current series. The knowledge unlocked is architectural judgment applicable to every Angular app you build going forward.

Production readiness: Completing Topic 59 means you have a complete, production-grade understanding of the NgRx ecosystem — from core concepts through Entity, Component Store, SignalStore, Router Store, Testing, and architectural best practices. You are equipped to lead NgRx architecture decisions on a production Angular team.

Section 14 — Common Interview Questions

Q1 (Mid): How do you decide whether state belongs in the global NgRx store, NgRx SignalStore, or a local Angular Signal?

Strong answer: Global NgRx store — shared across multiple features, persists across navigation, needs DevTools audit trail, or drives complex async Effect chains. SignalStore or Component Store — complex local feature state with 3+ slices or async side effects, scoped to one component. Local signal() — simple 1-2 value UI state that nothing outside the component needs. Decision question: 'Does anything outside this component need this state, today or in the foreseeable future?'

Weak answers miss: The SignalStore middle ground.

Q2 (Senior): What is the difference between an action as an event versus a command, and why does it matter architecturally?

Strong answer: Event describes what happened ([Orders Page] Order Delete Clicked) — past tense, fact-based. Command describes what to do ([Orders] Delete Order) — imperative. Events decouple the dispatcher from the handler. One event can be handled by multiple reducers AND multiple effects simultaneously — impossible with command-style actions without many fine-grained commands. DevTools readability: filtering by [Orders Page] shows every user interaction; filtering by [Orders API] shows every server response.

Weak answers miss: The one-event-multiple-handlers architectural benefit.

Q3 (Senior): A junior developer says 'I've added NgRx to our new Angular project because we want to follow best practices.' What questions do you ask to evaluate this decision?

Strong answer: How many features share state? How many developers? Does the app need an audit trail? Are there complex async sequences? What's the team's RxJS experience? If not justified — NgRx can be removed incrementally; replace with services + Signals for simple cases. Modern alternative: Angular Signals + SignalStore covers the majority of state management needs without NgRx's boilerplate. Avoid the 'we might need it later' trap — YAGNI applies to NgRx.

Weak answers miss: The honest acknowledgment that NgRx can be the wrong tool.

Section 15 — Anti-Patterns to Avoid in Production

Anti-Pattern 1 — The 'everything in the store' anti-pattern

isNavMenuOpen, activeTabIndex, modalOpen, accordionExpandedIds, formStep, tableScrollPosition all added to the global store. DevTools logs thousands of [Nav] Menu Opened actions per session, hiding meaningful changes. Every route carries and resets ephemeral UI state. Onboarding requires understanding 200 state slices when 180 are ephemeral toggles. Correct: local signal() for ephemeral UI state; global store for shared, persistent state only.

Anti-Pattern 2 — Dispatching actions inside Effects with tap but missing dispatch: false

```
// WRONG – creates infinite dispatch loop
trackAnalytics$ = createEffect(() =>
  this.actions$.pipe(
    ofType(OrdersPageActions.pageOpened),
    tap(() => this.store.dispatch(AnalyticsActions.pageViewed())),
    // Missing { dispatch: false } – the original action also gets
    // dispatched as the effect's output, triggering it again -> infinity
  )
);

// Missing second argument: { dispatch: false }

// CORRECT – { dispatch: false } for side-effect-only effects
trackAnalytics$ = createEffect(() =>
  this.actions$.pipe(
    ofType(OrdersPageActions.pageOpened),
    tap(() => this.analyticsService.track('orders_page_viewed')),
  ),
  { dispatch: false }
);
```

Anti-Pattern 3 — Inline selector lambdas in store.select() calls

```
// WRONG – inline lambda, no memoisation
orders$ = this.store.select(state => state.orders.list);

// CORRECT – named selector, fully memoised
// In orders.selectors.ts:
export const selectOrderList = createSelector(
  selectOrdersState, state => state.list
);
```

```
// In component:
orders = this.store.selectSignal(selectOrderList);
```

Inline lambda creates a new function reference on every call — NgRx cannot memoise anonymous functions. Every `store.select(state => state.slice)` creates a new Observable subscription with no memoisation. With `OnPush`, components re-render on every store action. With hundreds of components doing this, CD performance degrades significantly.

Section 16 — Mental Model Summary

Core concept: NgRx best practices are seven laws — state in the right layer, actions as past-tense events, all side effects in Effects, correct operator selection for each write pattern, `catchError` inside inner Observables, feature state lazy-registered, and `OnPush` on every connected component — every production NgRx bug traces back to violating exactly one of these.

	Global NgRx Store	SignalStore	Local signal()
Scope	App-wide	Component + descendants	Single component
Lifetime	App session	Component lifetime	Component lifetime
Best for	Shared/persistent data	Complex local UI (3+ slices)	Simple flags/toggles
Write API	Dispatch actions	<code>withMethods()</code> + <code>patchState()</code>	<code>signal.set()</code> / <code>update()</code>
DevTools	Full time-travel	Limited	None
Boilerplate	High	Medium	Minimal

Most important rule: Never use `switchMap` for write operations (POST, PUT, DELETE) — it silently cancels in-flight requests on any new emission, causing data corruption with no error message and no console warning.

— END OF TOPIC 59 — END OF ANGULAR LEARNING SERIES TOPICS 54-59 —